3) [FightingEntropy(π)]://Infrastructure Deployment System

[3A]: Ambitious Automation

We begin by transitioning into a scene from:

10/25/2021	Demonstration - Ambitious Automation	1h 56m	https://youtu.be/1E-3POI29Jo

...where a prior version of [FightingEntropy(π)] was once used to (deploy/configure/manage/secure) a pair of Hyper-V virtual machines:

(VM1): OPNsense v21.7 Noble Nightingale
(VM2): Windows Server (2019) – Datacenter

If you would like to see that video, it's accessible, but <unlisted>. However, I've taken notes on the process as demonstrated in that link, and have laid it all out below. It will coalesce into other criteria as the chapter goes.

Note: this video was published about 1Y before <generative AI> like <ChatGPT> was being used to write scripts and programming, which means that I wrote all this stuff the *old-fashioned* way, without *vibe-coding* it.

[3B]: Virtual Machines (1 + 2)

These (2) virtual machines are co-dependent, which means that the <VmController> has to bootstrap both VMs <successfully>, in order for each of them to be configured successfully. They cannot do this if any part of the process is broken.

What I did in this scenario where I threaded them together, is not a great idea in most cases, as there are race conditions throughout the script. The measurement I used at the time to determine whether a process on a guest machine was completed, was continuously checking idle CPU time. Not the best approach.

However, it made sense to use this approach in this particular circumstance.

(VM1): will have (2) virtual network adapters, one to connect to the internet, the other to the private network.

entry-01.jpg

This process is also called (NAT/Network Address Translation), but in this instance, it is just a standard issue (gateway/router) that we'll be configuring after installation using the (WMI/Windows Management Instrumentation) classes dealing with virtualization.

(VM2): will have (1) virtual network adapter to connect to the network that is shared internally by the gateway, the LAN. The Local Area Network will be joined together through the virtual switch.

entry-02.jpg

If (VM1/OPNsense) is misconfigured, it won't provide internet for the server.
If (VM2/server) doesn't have internet, it can't download the (module dependencies/functions) used below.

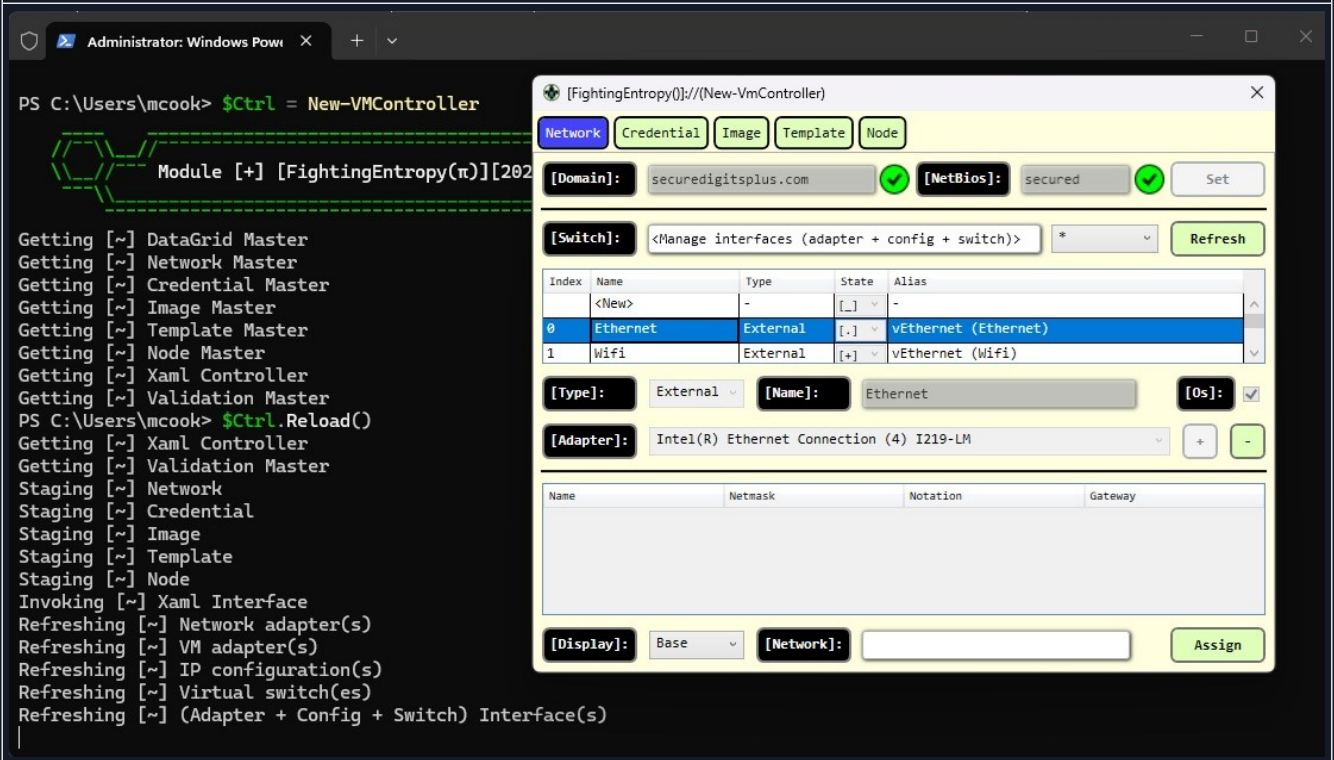
At which point, when both of them have been installed and configured, the rest of the network can be scripted to completion. At some point in this demonstration, we transfer images from a local SMB share to then install:

MDT	Microsoft Deployment Toolkit
WinADK	Windows Assessment and Deployment Kit
WinPE	Windows Preinstallation Environment

...to build a forward (FE/MDT) server that could be used to deploy child item systems, and control *other* machines attached to the <virtual switch>. This will work for (physical/virtual) machines, dependent on setup.

[3C]: Virtual Switches

It's worth noting that the <virtual switches> need to be created before making these virtual machines, otherwise they will have issues. At some point I will cover <virtual switch> creation, but not in this chapter. Here's a look at what <virtual switch> creation looks like in the [FightingEntropy(π)][2024.1.0] New-VMController GUI.



entry-03.jpg

So yeah, you don't need a fancy GUI to make them, but it helps to have a visual guide as to what you can do when creating them. <Virtual switches> don't work the same exact way that <regular switches> do, although

they come pretty close.

They occupy an existing physical NIC and share its' connection, and then they ride along through that interface.

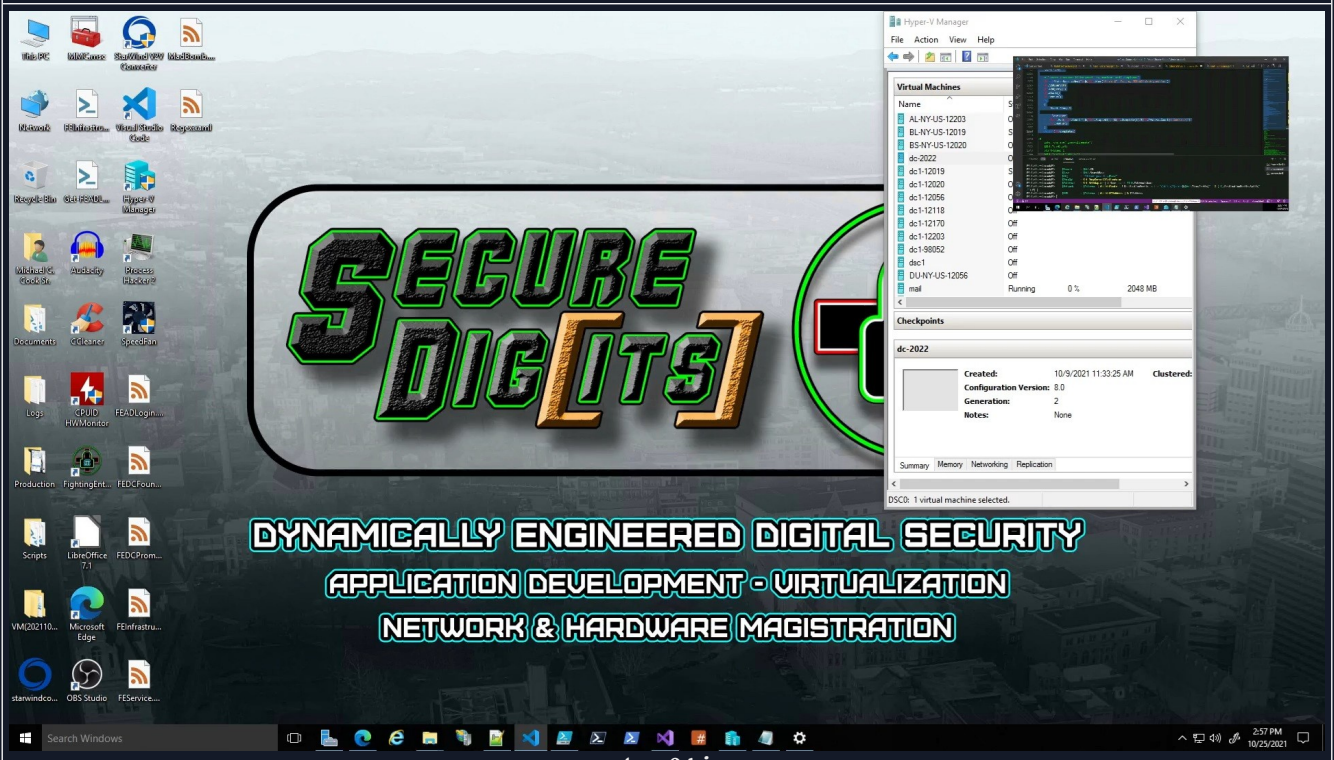
All virtual network activity has to go through these predefined links and adapters, so if you want (cluster/link aggregate) a bunch of (adapters/interfaces) together as (1), you would want to do that first. Then, build the <virtual switch> from there.

This poses SOME issues when you want the <virtual switch> to work alongside Cisco's spanning tree protocol type stuff. So, we may need to extend what the <virtual switch> configuration utility does at some point, for more integration.

When (VM1) is done installing, it will be configured with specific settings that were created by the [FightingEntropy(π)]://Infrastructure Deployment System. That part of IDS is not featured in this video.

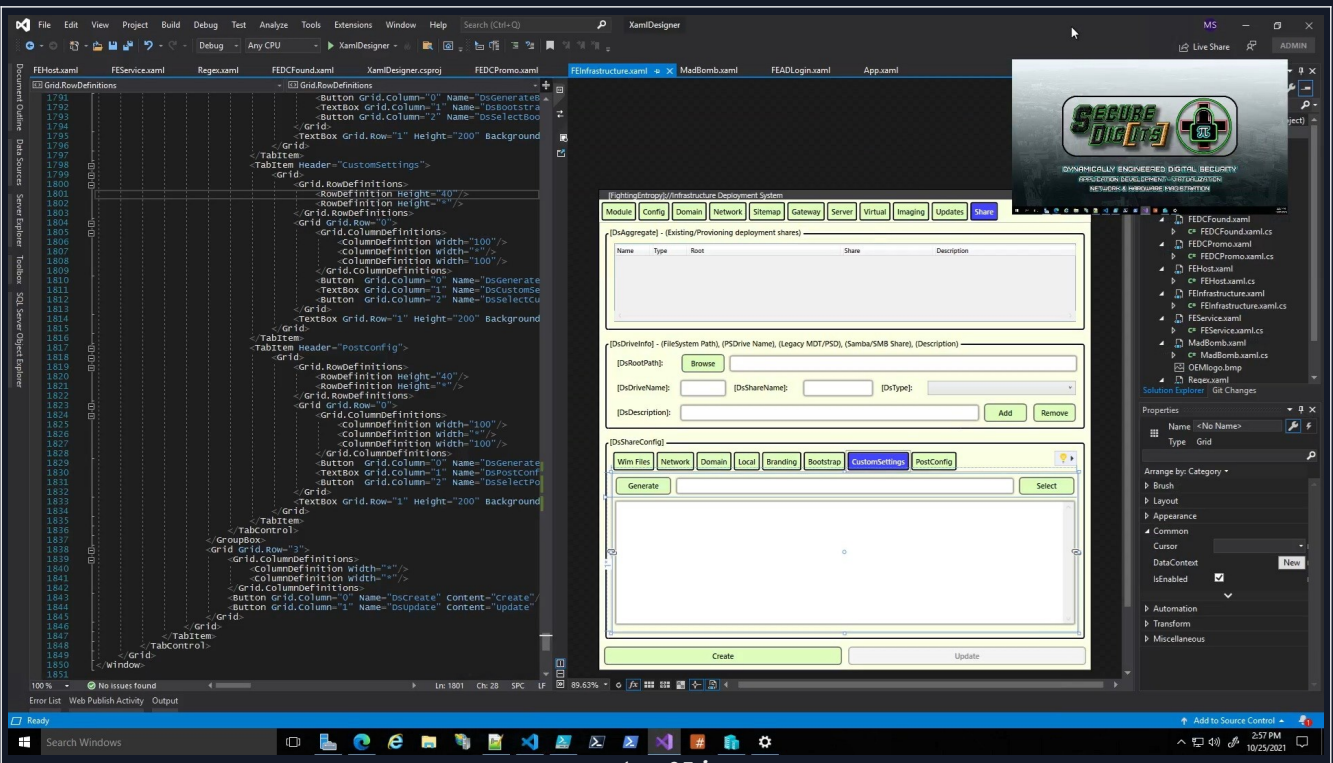
Same goes for (VM2), the templates were already created using IDS, but the <VmController> in IDS would eventually be its own function called New-VmController (shown in graphic above).

[3D]: Roll Demonstration



entry-04.jpg

If it looks like the environment of a cybercommando...? Think nothing of it.



entry-05.jpg

You can get a glimpse of some guy doing something with Visual Studio XAML Designer.

I have invested either hundreds or thousands of hours into writing Extensible Application Markup Language. I have also invested thousands of hours into writing the code behind that drives it, and work on both of them in tandem.

This is what the GUI for the [FightingEntropy(π)]://Infrastructure Deployment System looked like in 10/2021.

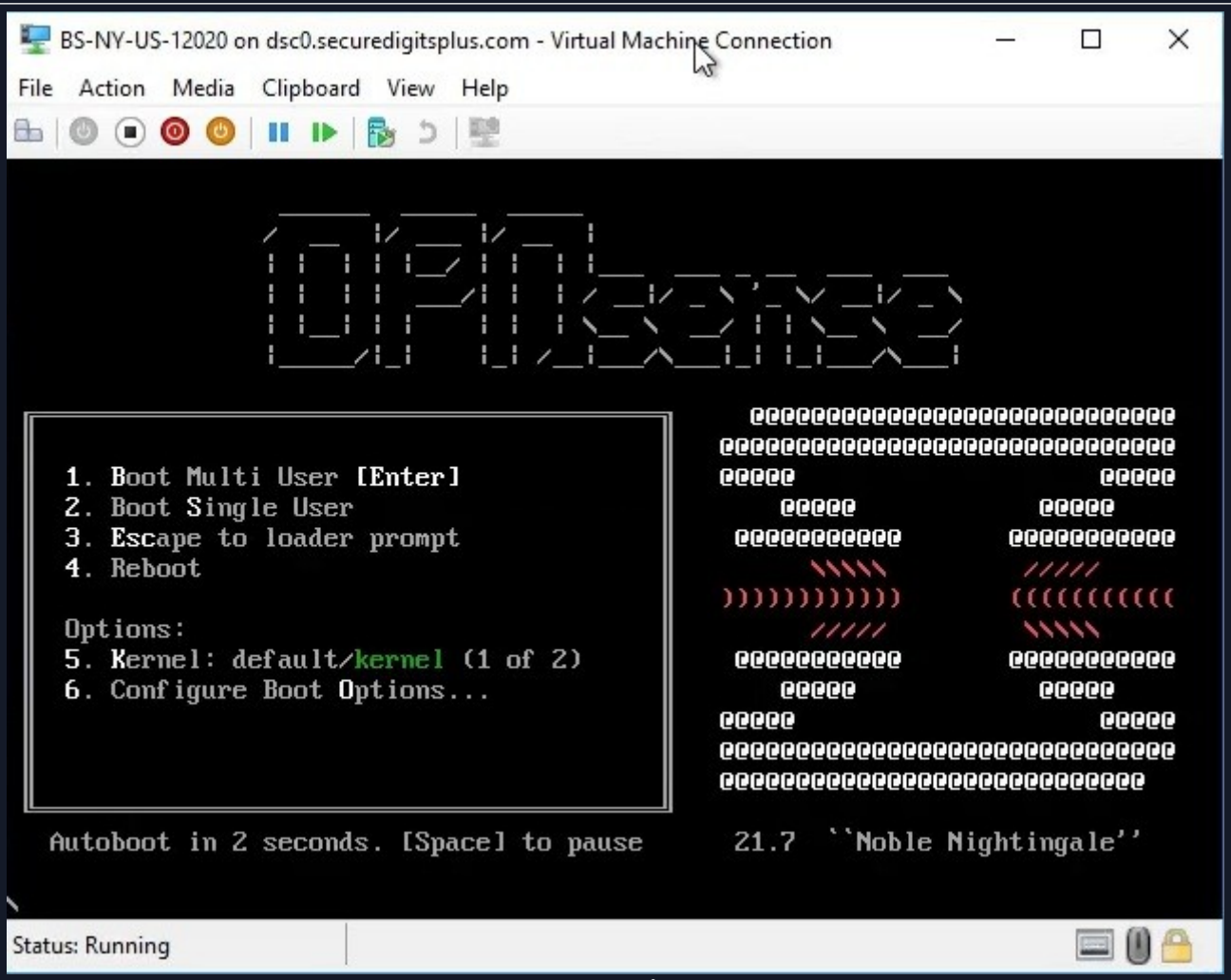
We will be making some serious style changes to the next iteration that will be virtualliy identical to Q3A-Live's <color scheme> and <style templates>.

```
$Domain     = $Mx1.CN
$Base       = $Mx1.SearchBase
$Cfg        = "CN=Configuration,$Base"
$DhcpOpt    = Get-DhcpServerV4OptionValue
$External   = Get-NetAdapter | ? Name -match $Inf.External.Name
$Network    = $External | Get-NetRoute | ? DestinationPrefix -match "(\d+\.){3}\d+\/$($Vm1.Item.Prefix)" | % { $_.DestinationPrefix.Split("/")[0] }
$DNS        = $External | Get-NetIPAddress | % IPAddress
```

entry-06.jpg

We see the so-called "cyber-commando" executing some basic PowerShell code.

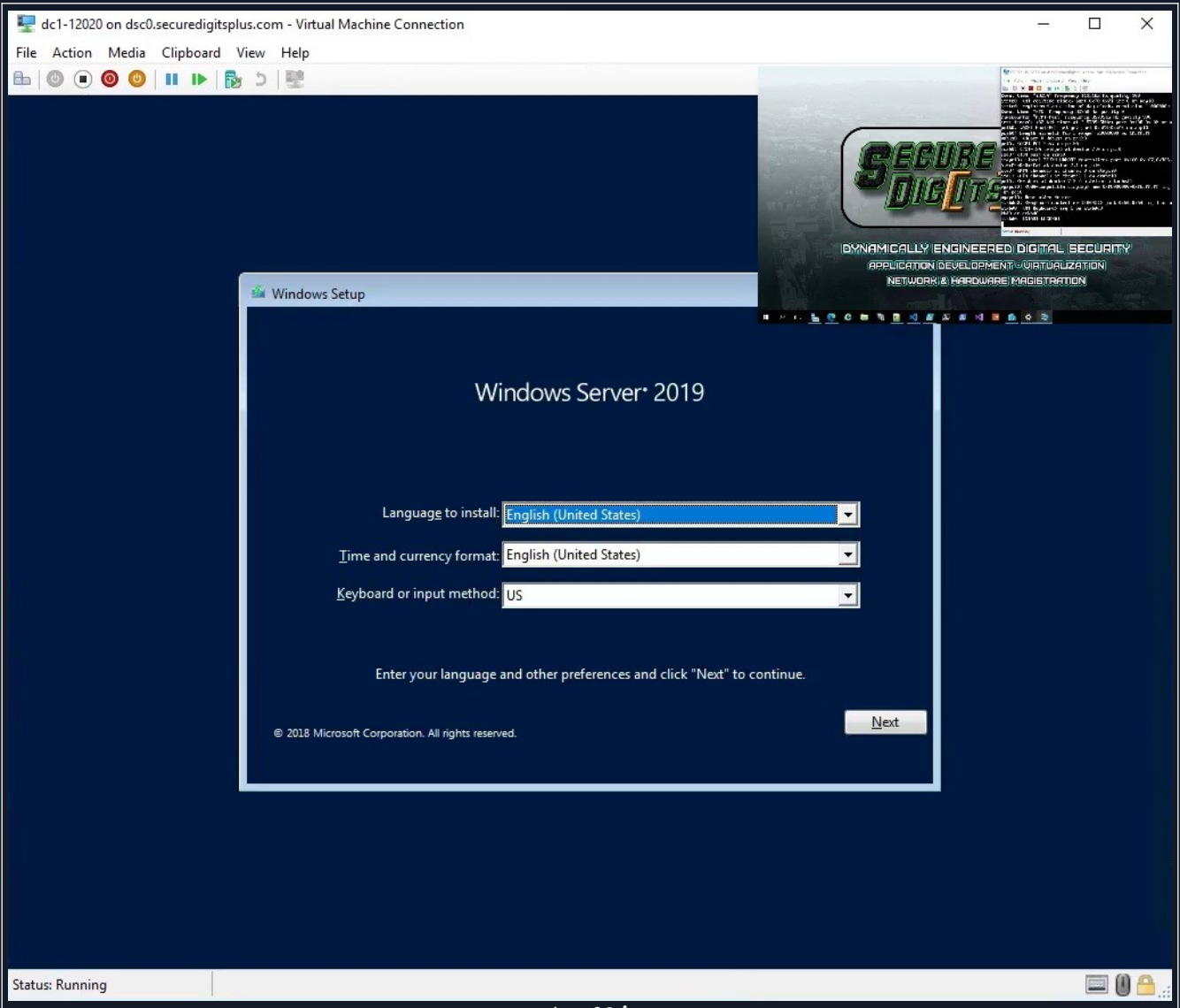
Looks like "Domain", "SearchBase", "CN=Configuration,$Base" DhcpServer, NetAdapter, NetRoute, NetIPAddress. I'd be willing to bet it's all for a good reason.



entry-07.jpg

Alas, we see what appears to be OPNsense v21.7 “Noble Nightingale”.
They like to use names with alliterative titles based on an animal.

Some people may not realize that the word alliteration is “all iteration”.

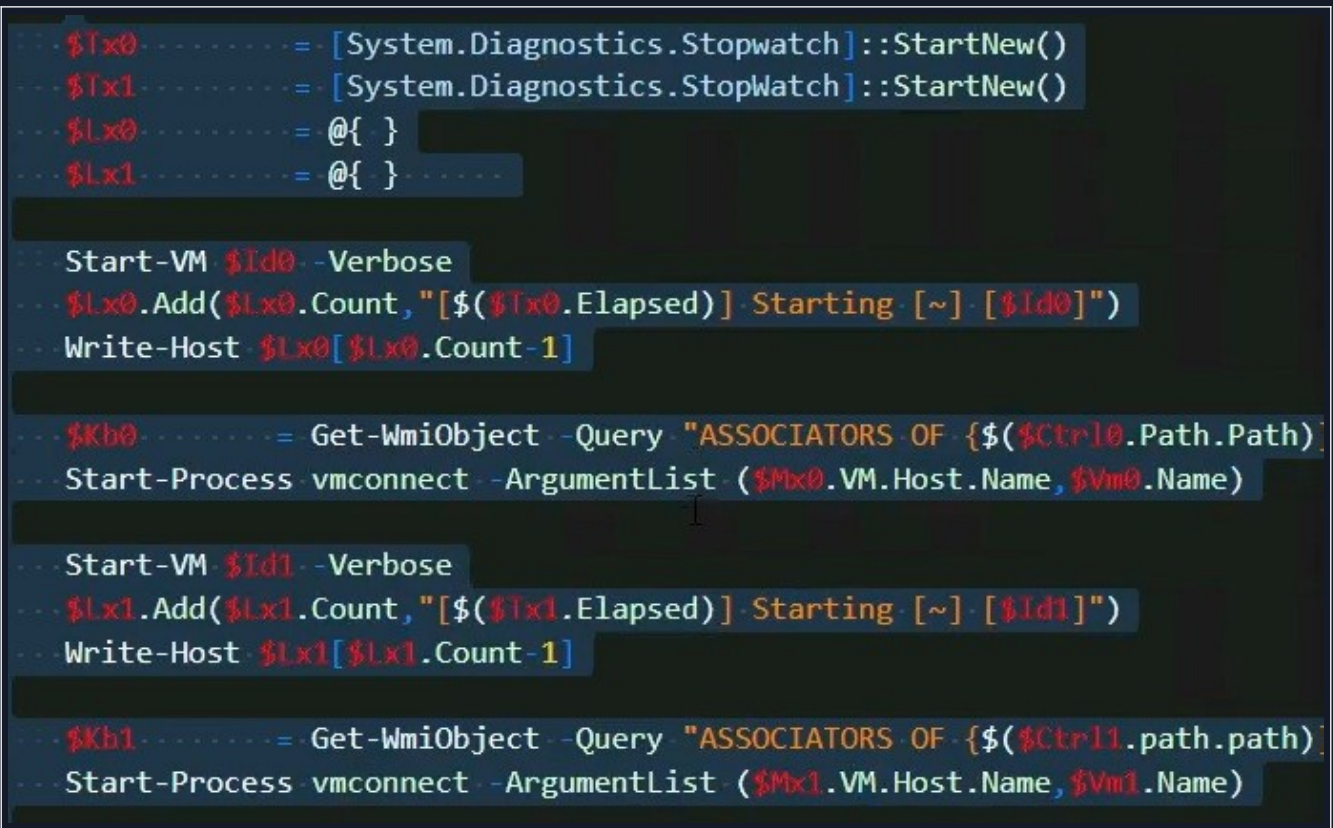


entry-08.jpg

Seconds later, this appears as well. The fearsome “Windows Setup (Windows Server 2019)” dialog.

After some additional commands, these (2) systems will be off to the races.

[3E]: Review Scriptblock #1



entry-09.jpg

The first set of arguments that had gotten both of those machines to boot into the installation environment.

The script above is reflected again below, with the same color-coded syntax. I do not have the script that was running on that system in the above video at that time, but I have other scripts that showcase the same logic.

```
# [What it once was]
$Tx0 = [System.Diagnostics.Stopwatch]::StartNew()
$Tx1 = [System.Diagnostics.Stopwatch]::StartNew()
$Lx0 = @{ }
$Lx1 = @{ }

Start-VM $Id0 -Verbose
$Lx0.Add($Lx0.Count, "[$($Tx0.Elapsed)] Starting [~] [$Id0]")
Write-Host $Lx0[$Lx0.Count-1]

$Kb0 = Get-WmiObject -Query "ASSOCIATORS OF {$($Ctrl0.Path.Path)} WHERE resultClass = Msvm_Keyboard" -NS Root\Virtualization\V2
Start-Process vmconnect -ArgumentList ($Mx0.VM.Host.Name, $Vm0.Name)

Start-VM $Id1 -Verbose
$Lx1.Add($Lx1.Count, "[$($Tx1.Elapsed)] Starting [~] [$Id1]")
Write-Host $Lx1[$Lx1.Count-1]

$Kb1 = Get-WmiObject -Query "ASSOCIATORS OF {$($Ctrl1.Path.Path)} WHERE resultClass = Msvm_Keyboard" -NS Root\Virtualization\V2
Start-Process vmconnect -ArgumentList ($Mx1.VM.Host.Name, $Vm1.Name)

# [What it became]
$Vm.Control  = Get-WmiObject MSVM_ComputerSystem -NS Root\Virtualization\V2 | ? ElementName -eq $Vm.Name
$Vm.Keyboard = Get-WmiObject -Query "ASSOCIATORS OF {$($Vm.Control.Path.Path)} WHERE resultClass = Msvm_Keyboard" -NS Root\Virtualization\V2

# [What it will be]
$Module.Platform.Wmi('\\.\root\virtualization\v2:MSVM_ComputerSystem.CreationClassName="Msvm_ComputerSystem"') | ? Name -eq $Vm.Name
```

The logging, time, and console objects have been replaced by another system in C#.
The WMI queries have been replaced by a much more responsive and capable WMI engine in C#.

```
    Do
    {
        $Item = Get-VM -Name $Id1
        Start-Sleep -Milliseconds 100
    }
    Until ($Item.Uptime.TotalSeconds -ge 2)
    $Kb1.TypeKey(13)

    Start-Sleep 20

    0..2 | % { $Kb1.TypeKey(9); Start-Sleep -M 100 }
    $Kb1.TypeKey(13)
    Start-Sleep 1

    $Kb1.TypeKey(13)
    Start-Sleep 20

    40,40,40,40,9,13 | % { $Kb1.TypeKey($_); Start-Sleep -M 100 }; Start-Sle
    32,9,13,9,13     | % { $Kb1.TypeKey($_); Start-Sleep -M 100 }; Start-Sle
    9,9,9,9,13       | % { $Kb1.TypeKey($_); Start-Sleep -M 100 }; Start-Sle
```

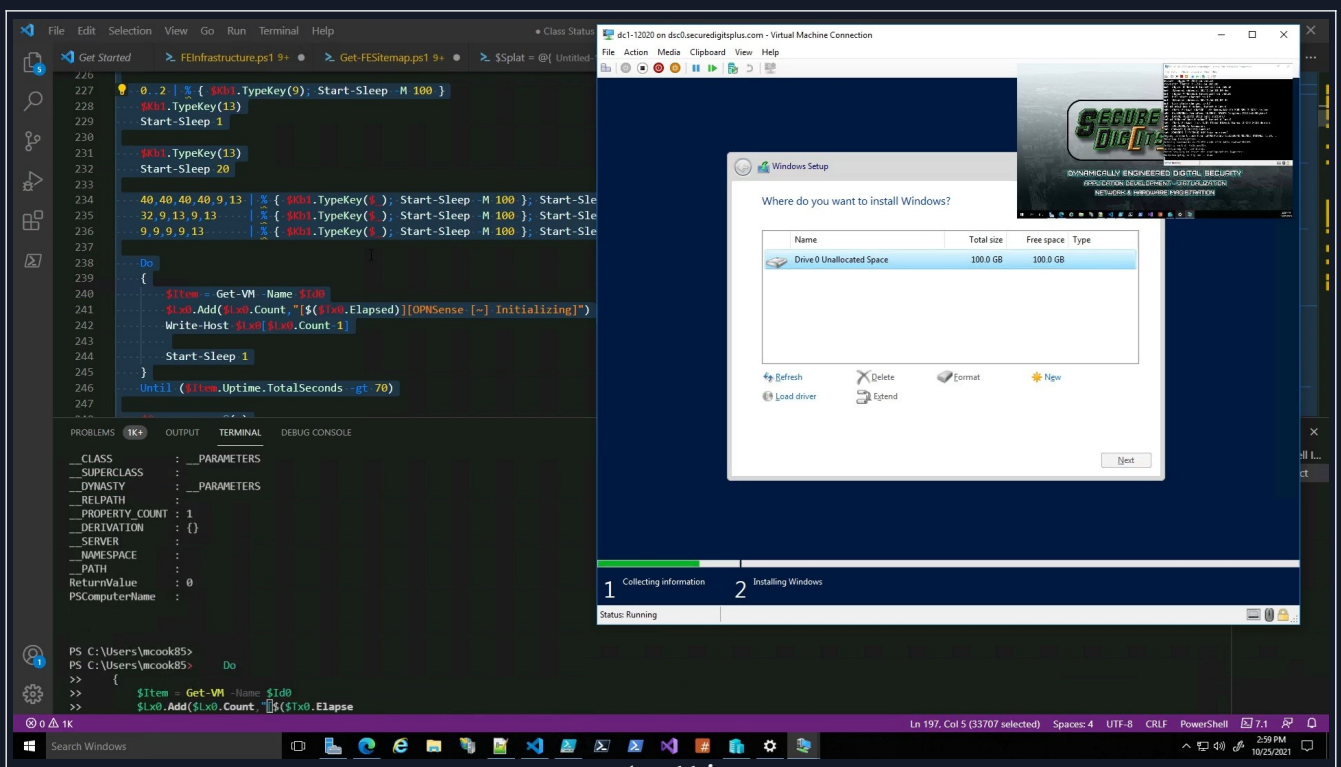
entry-10.jpg

And, the second. We can see from the last (2) screenshots, that the controller is starting the <virtual machine> via PowerShell, and entering parameters. Once the device is started, then the WMI object that controls the virtual machine can be grabbed, and then the process of installation is automated.

It also opens up a process to vmconnect, which opens the GUI to the particular virtual machine, but that is only necessary for demonstration purposes like this recording, that isn't required. We will be replacing all "Start-Process" (calls/references) with something specifically tailored around [System.Diagnostics.Process].

At this point, I had control mechanisms in place to manage the (creation/deletion) of <virtual machines>, however I would later come up with a better system for it, called "New-VmController".

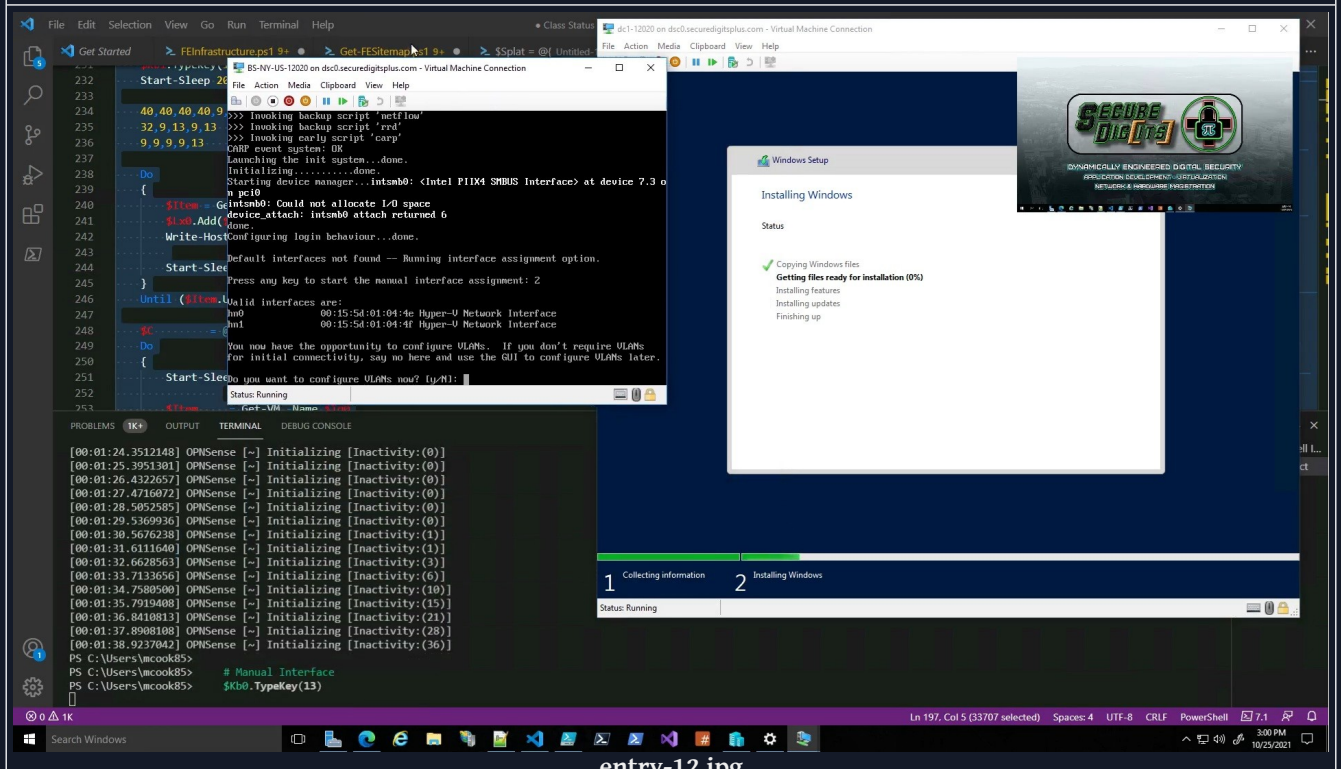
[3F]: Multi-Tasking Installations



entry-11.jpg

Here, the script for each machine is running in tandem. Like, this entire script was intertwining each process and having compartmentalized co-dependent <race conditions>, but they had to each successfully bootstrap in order for each of them to fully configure themselves.

It's actually a bit of a challenge to get multiple systems to (stage + script) themselves concurrently like I did.

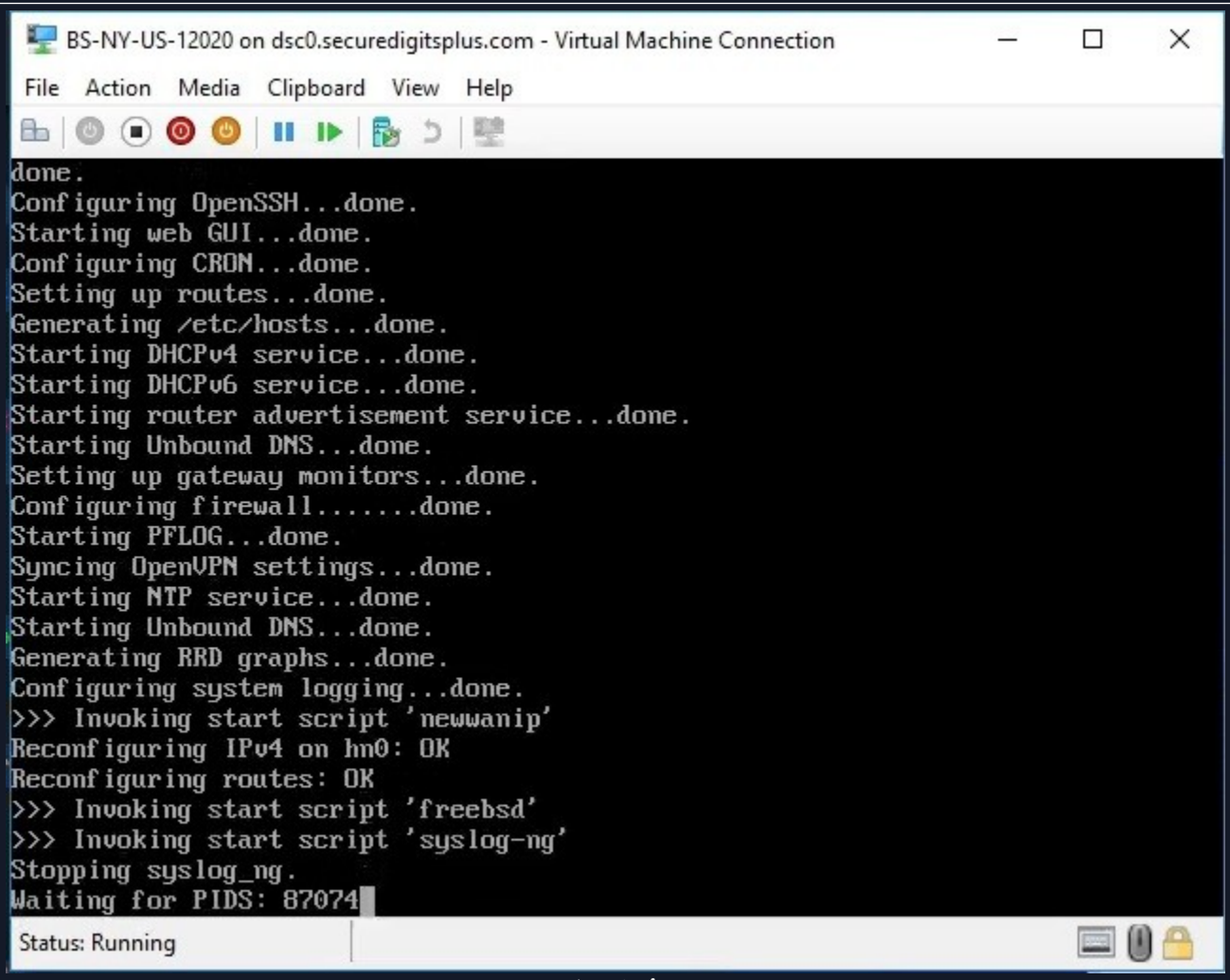


entry-12.jpg

So, here we see OPNsense finally pop up, and it is super quick in running these things on time, I had to go frame

by frame in the video to show the input in VS Code terminal $Kb0.TypeKey(13), and on the actual VM screen you can see “Do you want to configure VLANs now? [y/N]:” which means the script is pressing enter, which automatically selects the default “N” option.

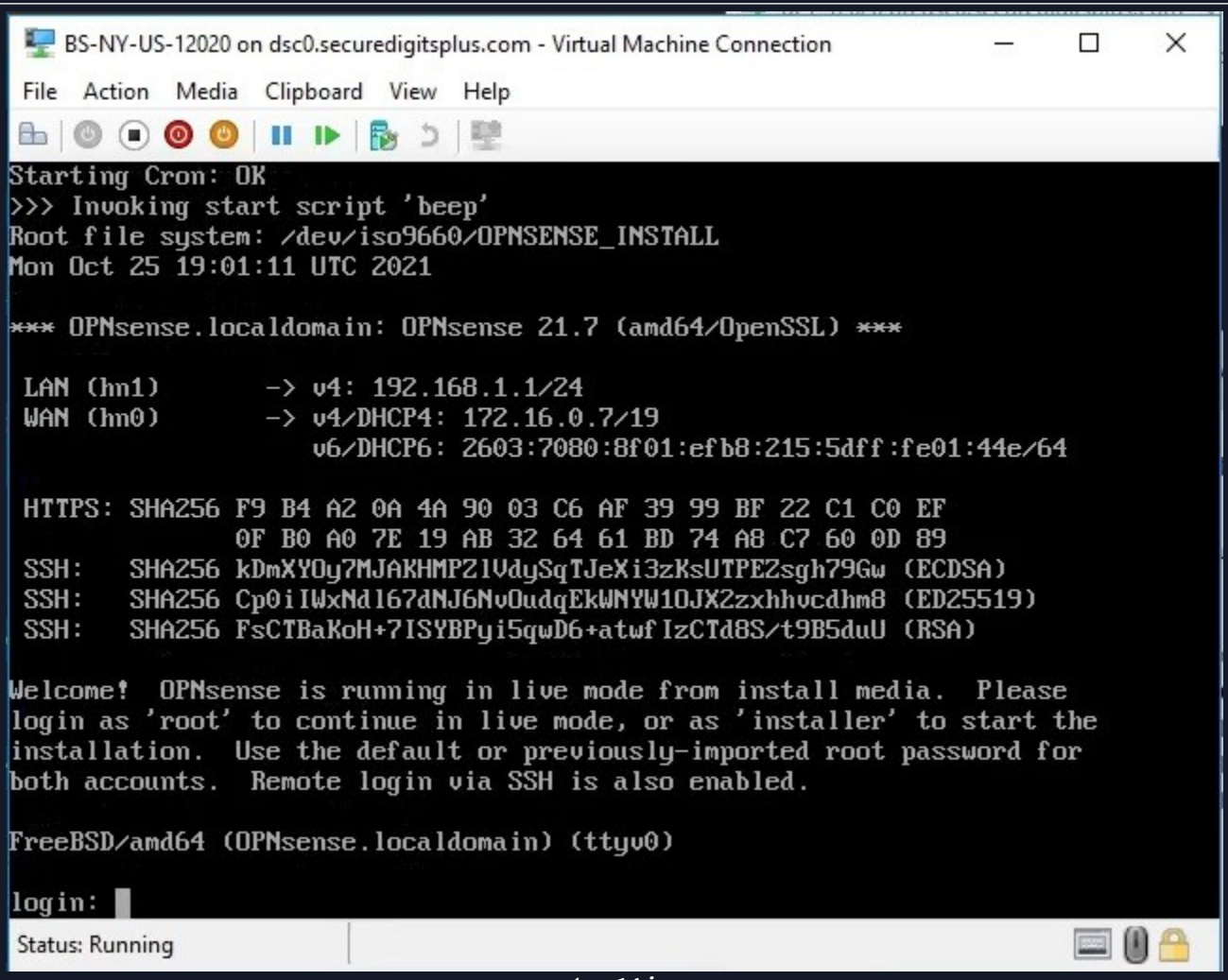
So, this process relies on control over the process that I would later implement better strategies for.



entry-13.jpg

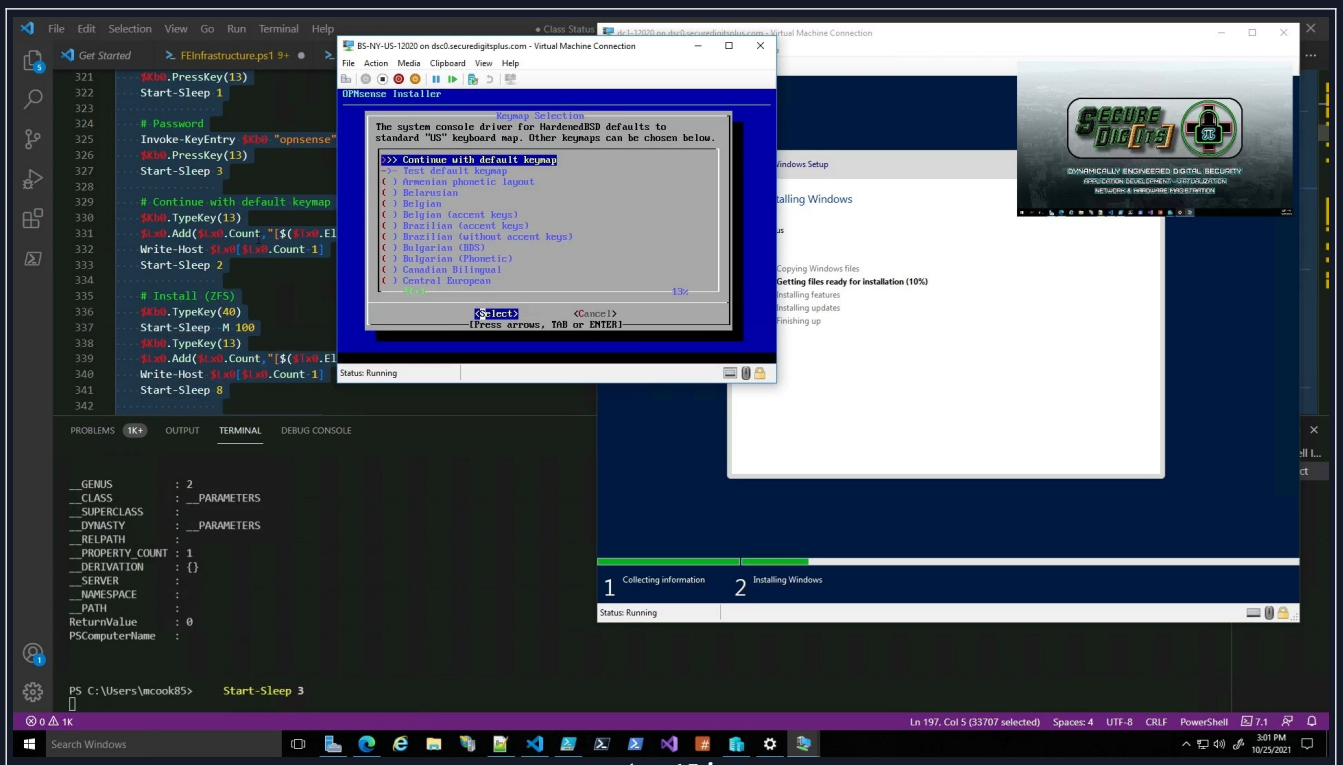
<OPNsense> will wind up looking like this as it is initializing. It’s worth noting that the tool I’m using is automatically generating those <hostnames> based on their <location>. So, I would enter the <zip code>, and then the program would look through the zip code database, select the correct info, populate various location tags that would occupy a certificate, and then tie all of that to the eventual WAN hostname.

[3G]: OPNsense Installation



entry-14.jpg

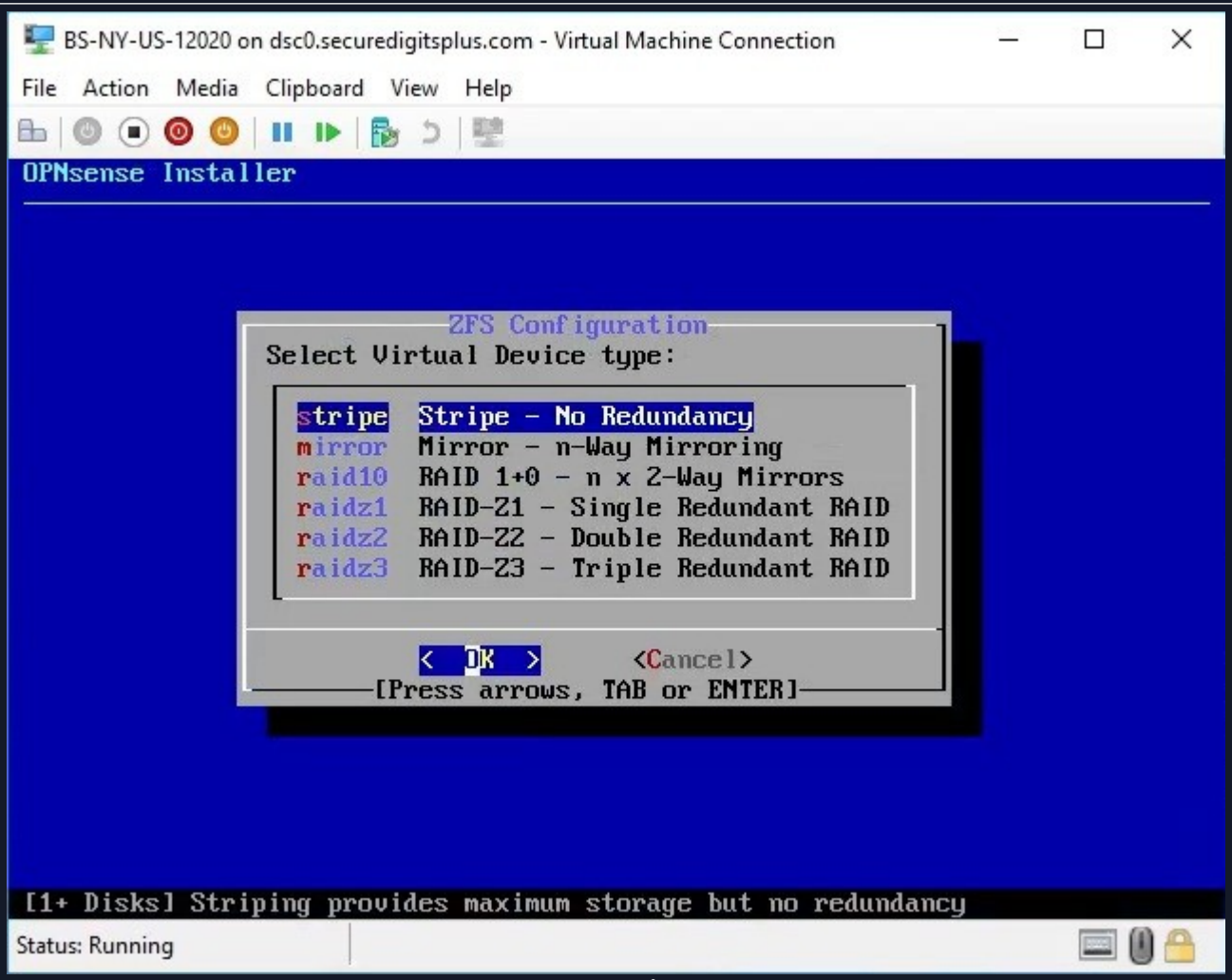
OPNsense ready to be (installed/configured). It will automatically use both default Hyper-V network adapters.



entry-15.jpg

I would occasionally run the demo and scroll along with the correct (location + order) of executing code.

At this point, it is about to install OPNsense, and once it is installed, the (*.iso) can be ejected, and then it runs off of the <virtual hard disk>.

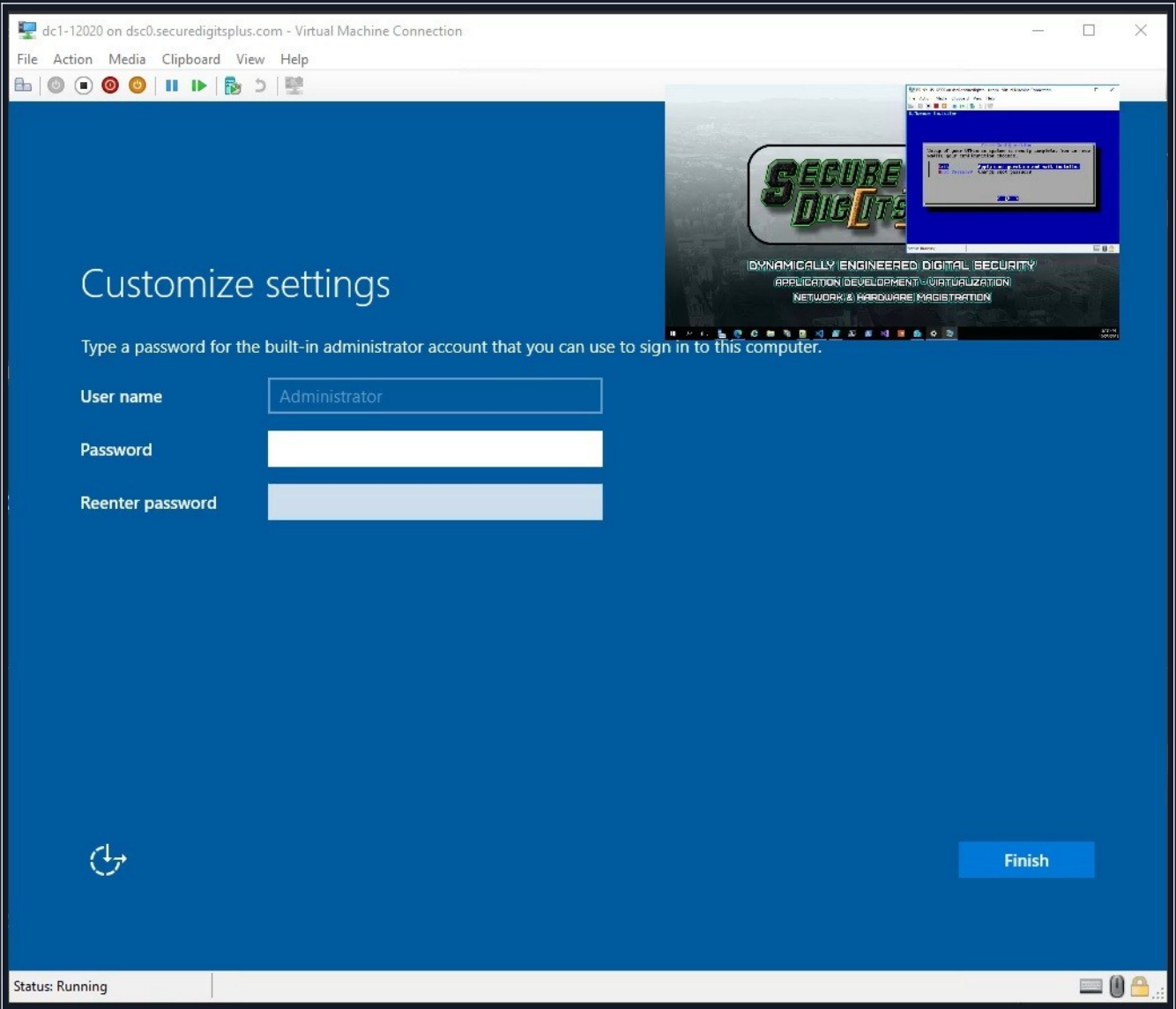


entry-16.jpg

One of the things it asks during setup, is whether you want to utilize some custom configurations. Suppose you wanted to run this virtual machine off of an existing RAID setup, then it would make sense to use “stripe”. Whereas if you weren’t on a RAID setup, then you could use RAID here.

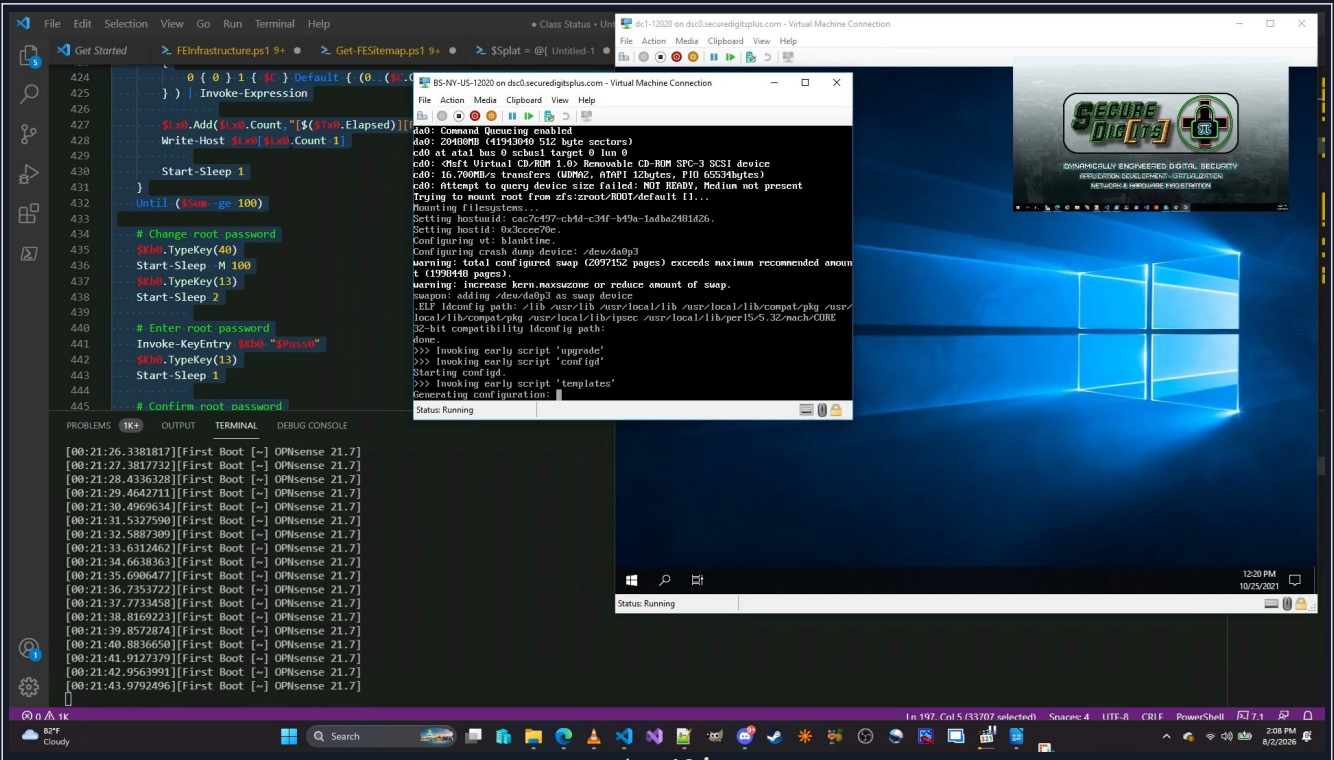
These are important to consider whether you’re building a gateway, or SQL server, or things of that nature, what is the device going to do, and how much will it get hammered? We have to design every possible solution here. I usually test it with the default config, but that would change depending on what else it gets used for.

[3H]: Server Installation Complete



entry-17.jpg

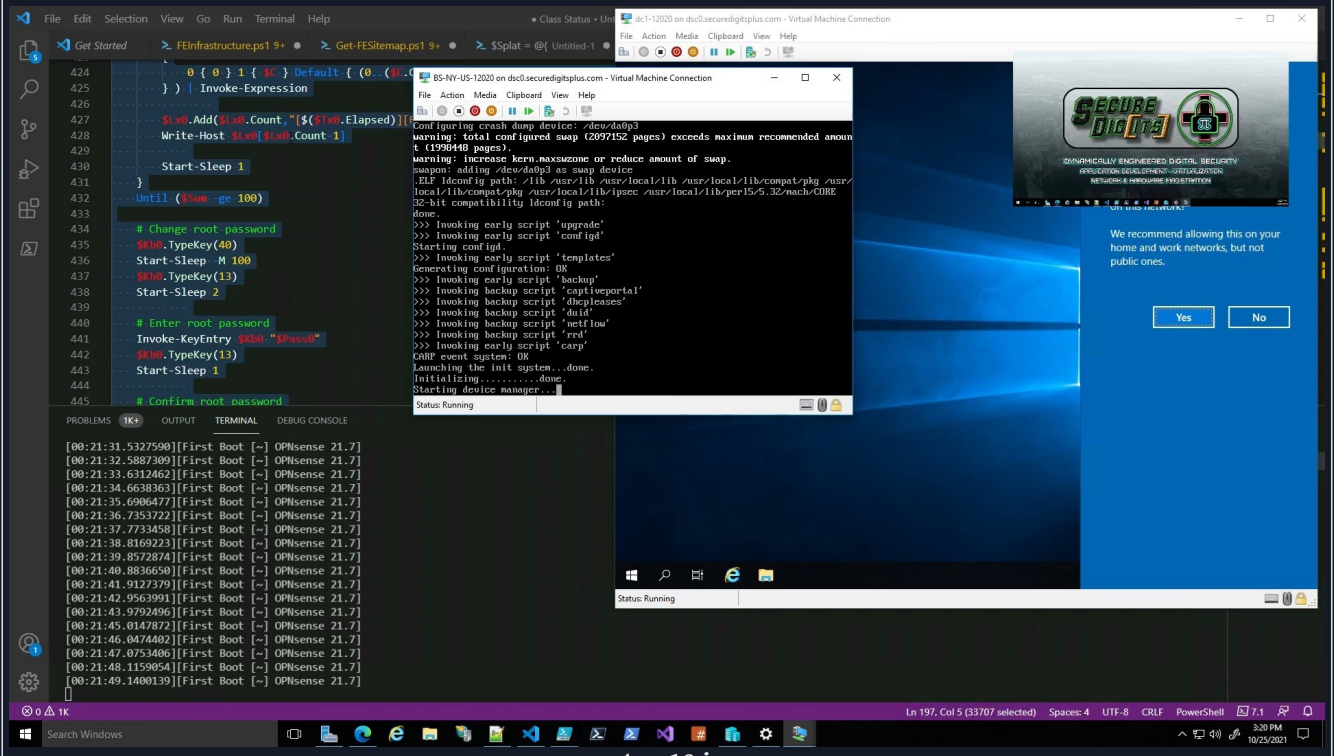
At this point, the server has completed the installation and now needs initial preparation. (FE) VMController would eventually save a series of credentials ahead of time and store them in encrypted form locally, and this process would be automated even more.



entry-18.jpg

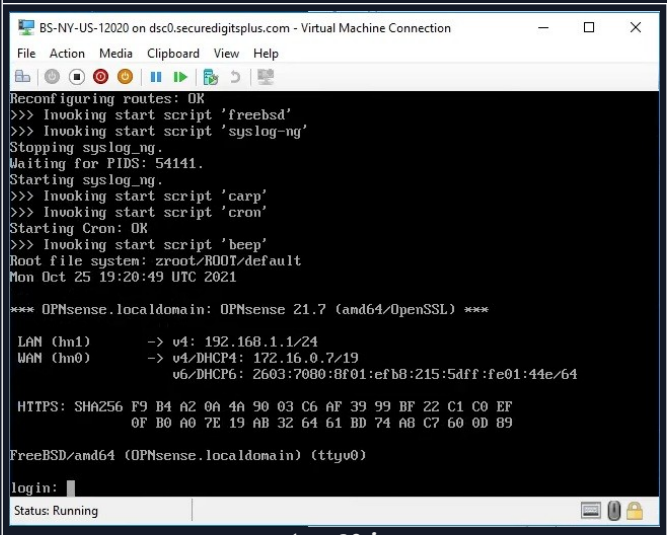
They both finish installing at approximately the same time, and thus, the OPNsense device can now be configured by the server, because the OPNsense gateway exposes a default private gateway address for initial browser configuration. You can use a configuration file as well, but this approach was using the WMI keyboard commands to control the Windows Server instance like what AutoHotkey would do, or something.

[3I]: Gateway Configuration (Phase 1)

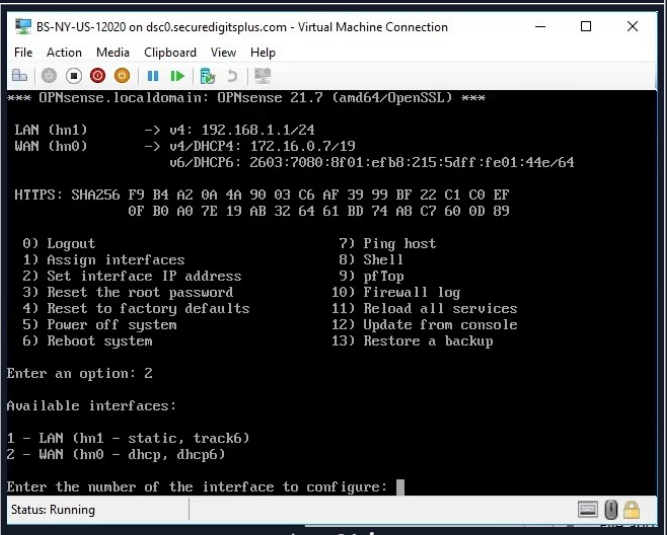


entry-19.jpg

Seconds later, you can see the OPNsense box is the outbound gateway for the server, and that the server sees it has a network connection, that means that OPNsense is providing a working connection immediately after installation. That means internet is accessible to the server via “NAT”. But- we need more configuration.

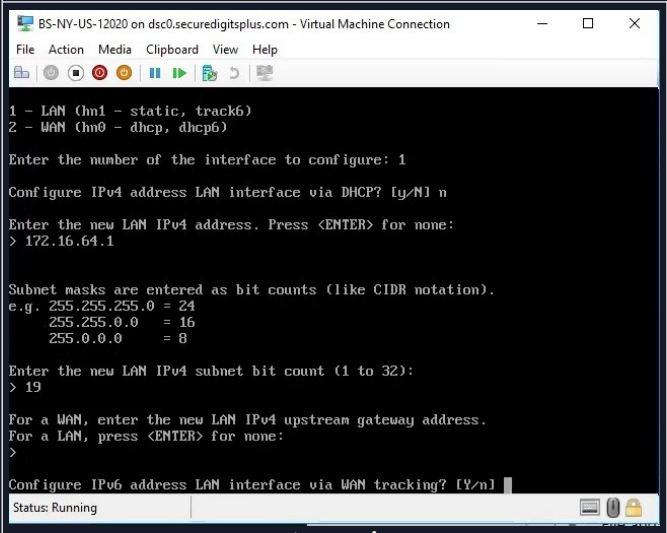


entry-20.jpg

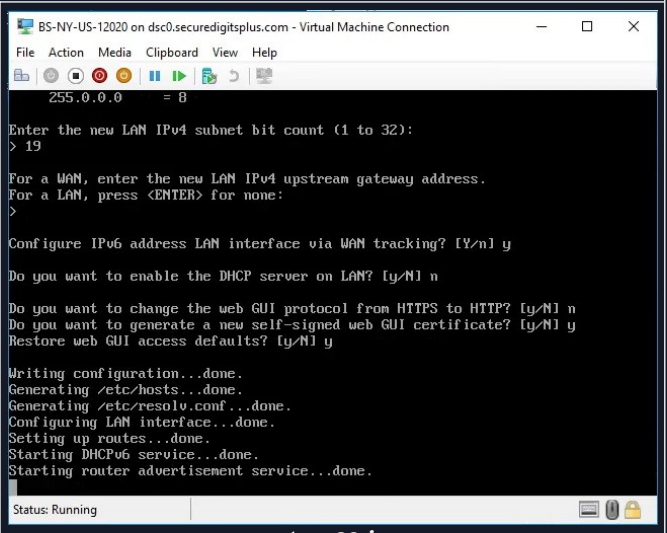


entry-21.jpg

At this point, the device could be configured using this interface, or, it can be configured over the web interface, but I wind up dialing-in a different LAN address, 172.16.64.1/19 (*CIDR notation = subnet mask 255.255.224.0*)



entry-22.jpg

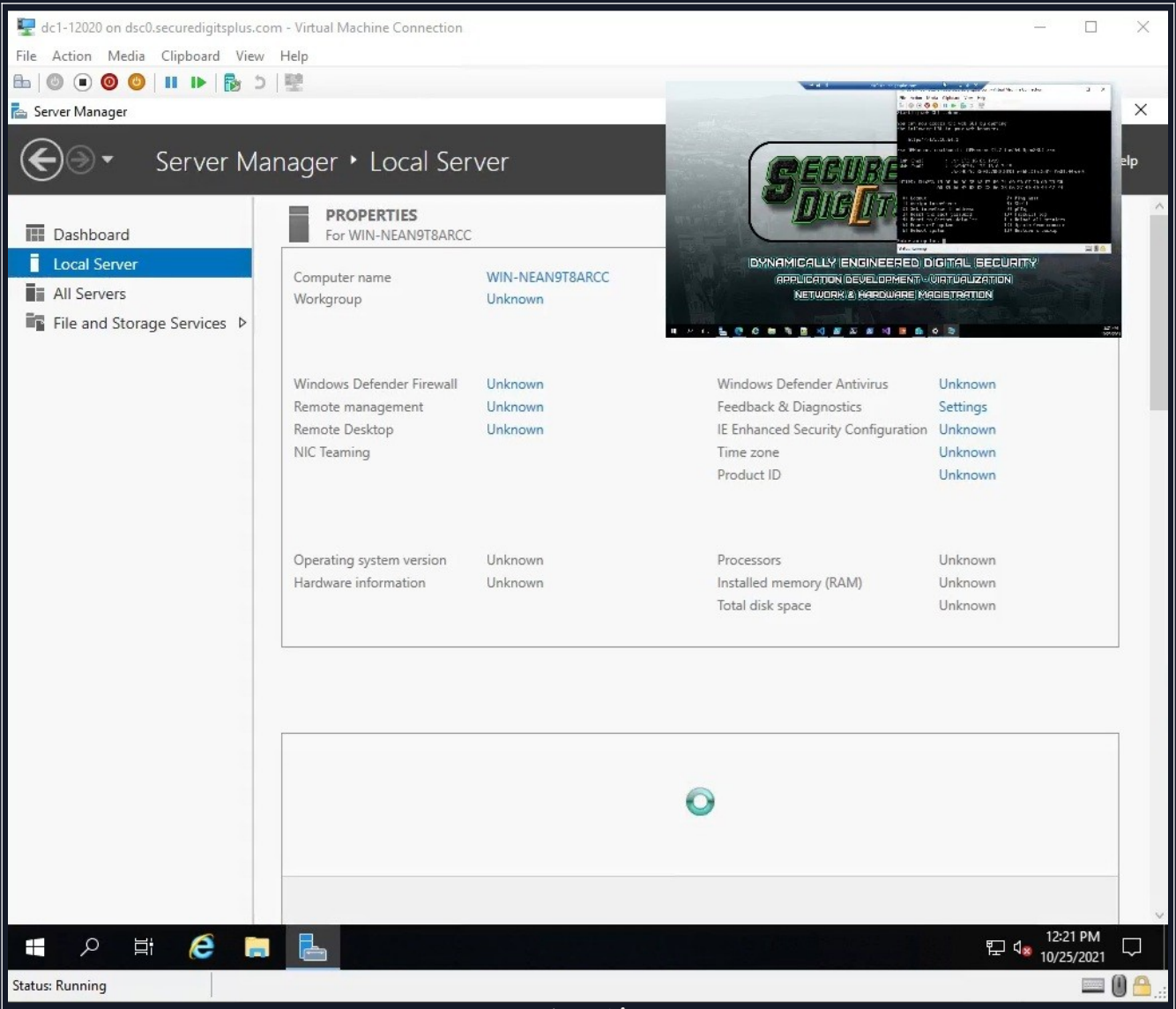


entry-23.jpg

Alright, so now the https://172.16.64.1:443 link will be up to configure the OPNsense gateway for *real.*

We’ll do this from the <Windows Server 2019> instance, using <Internet Explorer>. Newer editions have <Microsoft Edge>, but this version doesn’t come with <Edge> by default.

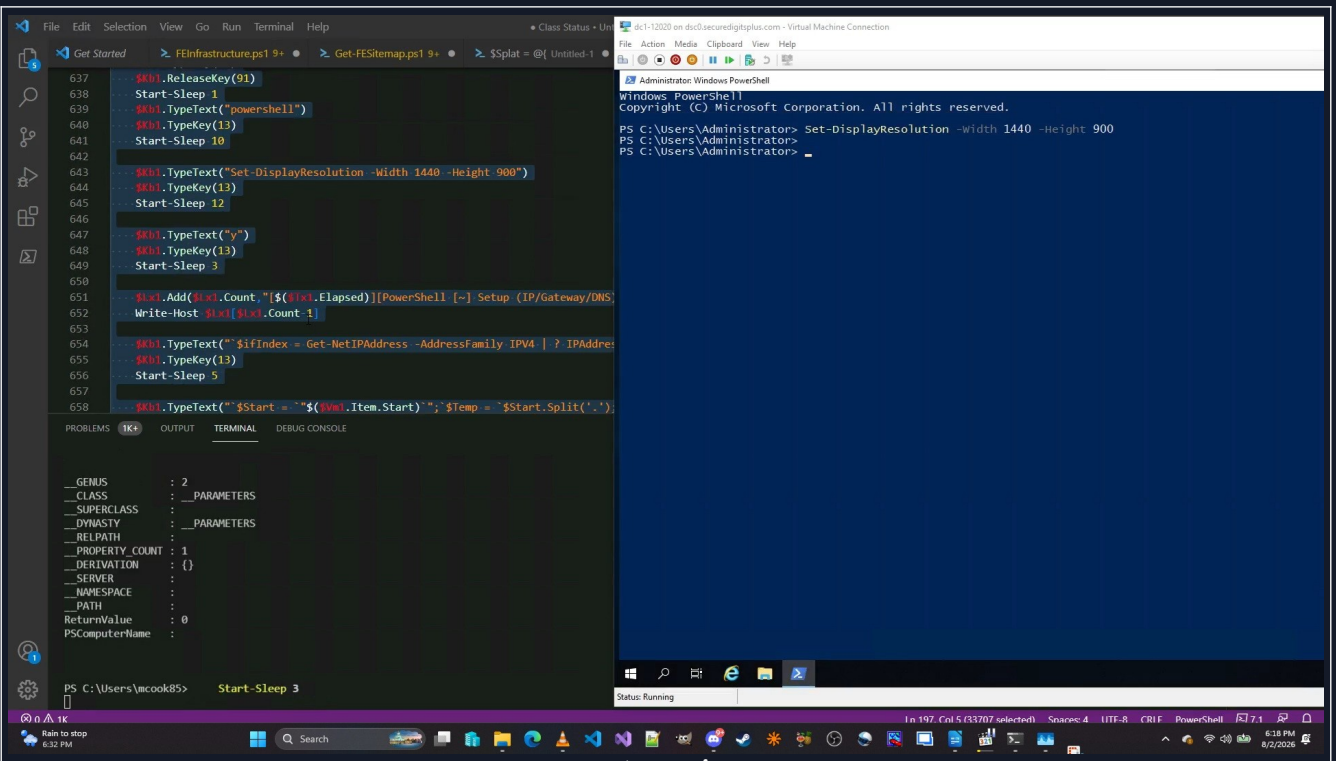
[3I]: Server Configuration (Phase 1)



entry-24.jpg

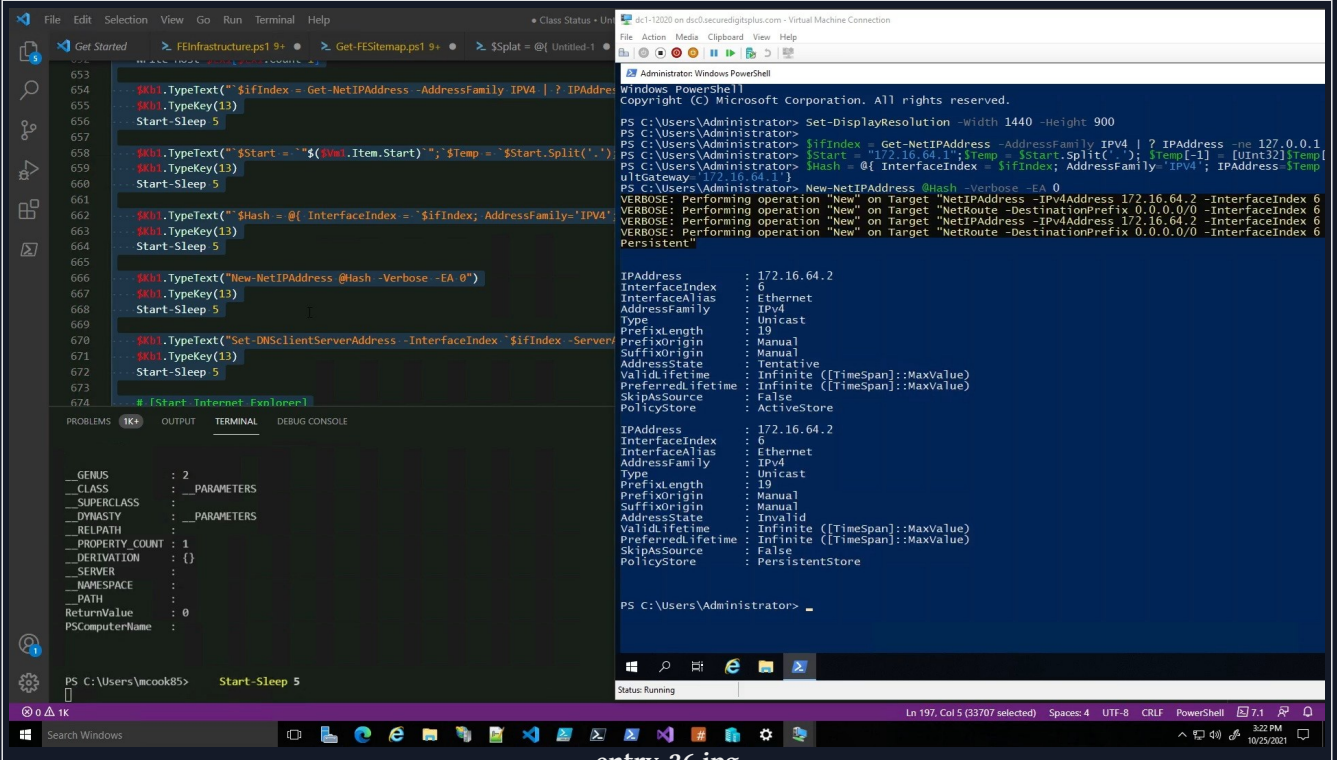
So HERE we have a snapshot of the Windows Server 2019 – Datacenter instance.
Computer name is "WIN-NEAN9T8ARCC" which is basically a random string of numbers and letters.

That has to be changed, but it doesn't need to be off the rip. We can configure the gateway first, and then start messing around with the configuration on this server.



entry-25.jpg

One of the first things we do as soon as the system is able to open an elevated PowerShell console, is to change the resolution. On Windows Server, there is a command “Set-DisplayResolution”, but that command doesn’t exist on Windows 10/11 by default. I have added a function that does this, based on some C# code I found on Reddit. I will wind up rewriting it at some point.



entry-26.jpg

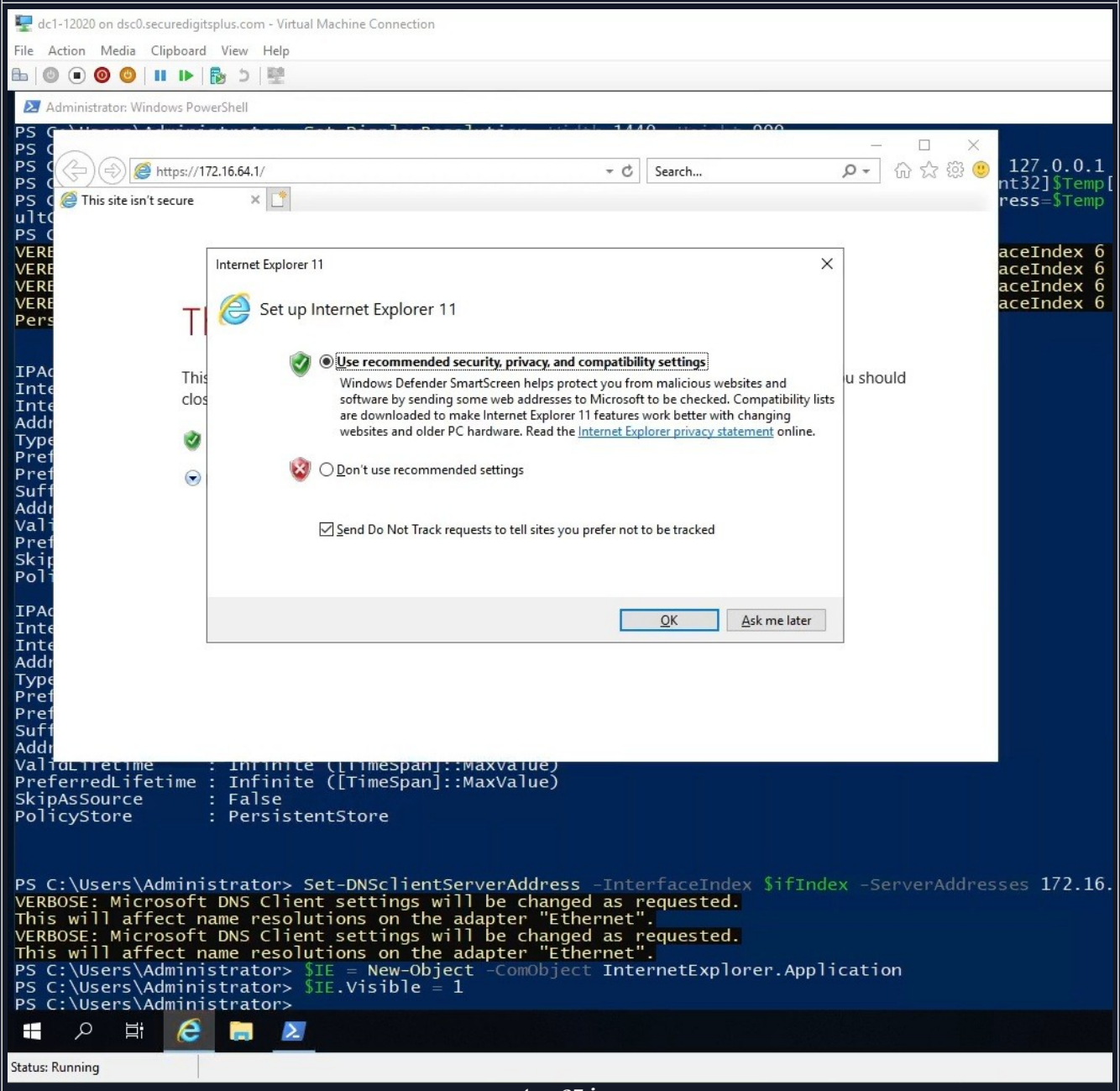
Another thing you will see, is how the $Kb1.TypeText(“String”) things are being typed in on the right. This was effectively replaced with Initialize-VmNode, which is a function that expects these networking number

templates and it automatically does all of this stuff that the script is passing over.

When the module is able to be installed, and the conditions match, it can override the existing settings and then change its configuration. Currently, that function is being rewritten in the C# conversion, so that it will work the same way for real machines, not just for virtual machines.

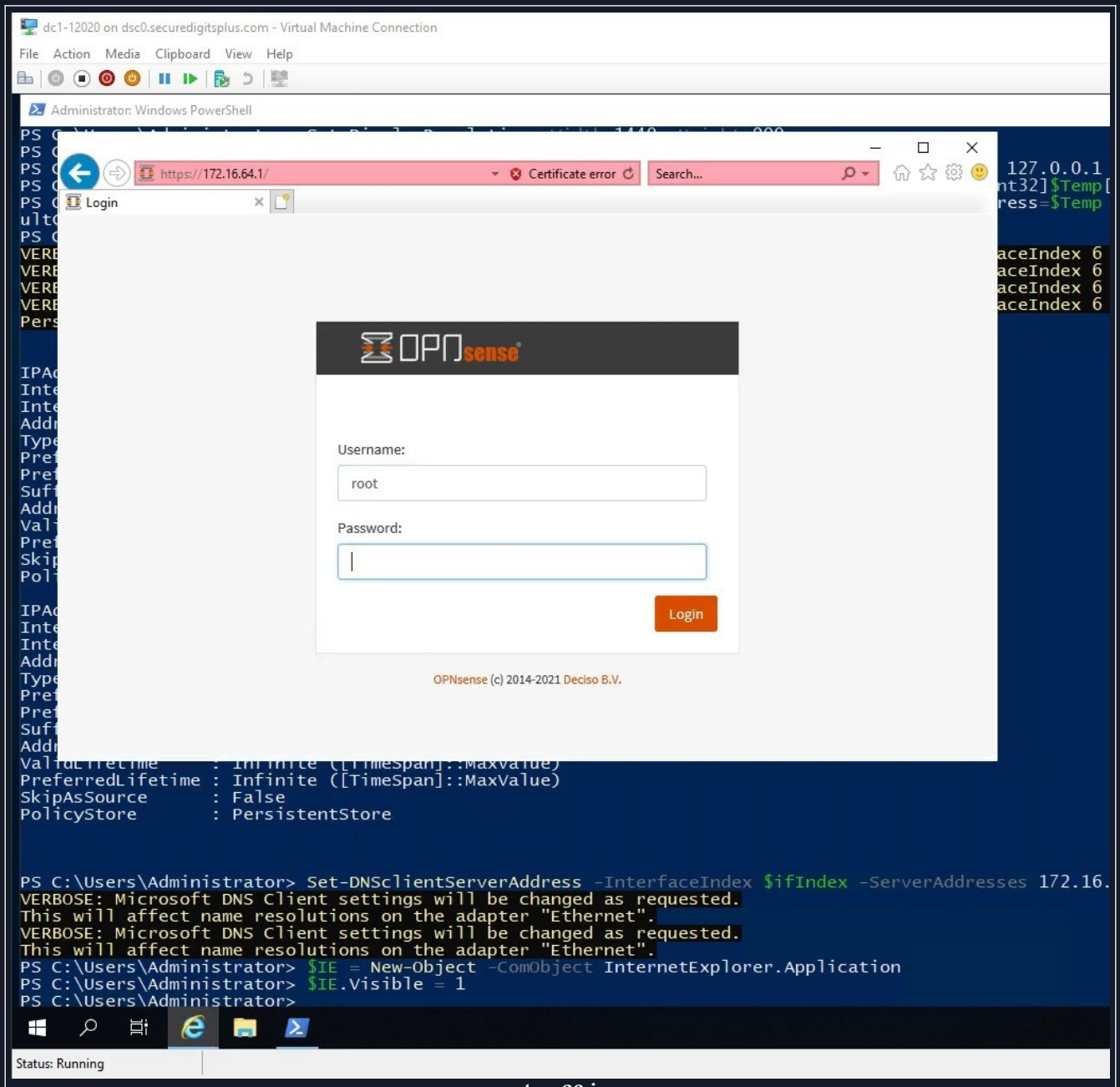
In fact, all of these scripts will be passed over via TCPSession objects, and it'll have them all in memory. So, as long as the script has been tested to work, it will process and log each script it has processed. Each "scriptlet" constitutes as its own "method".

[3K]: Gateway Configuration (Phase 2)



entry-27.jpg

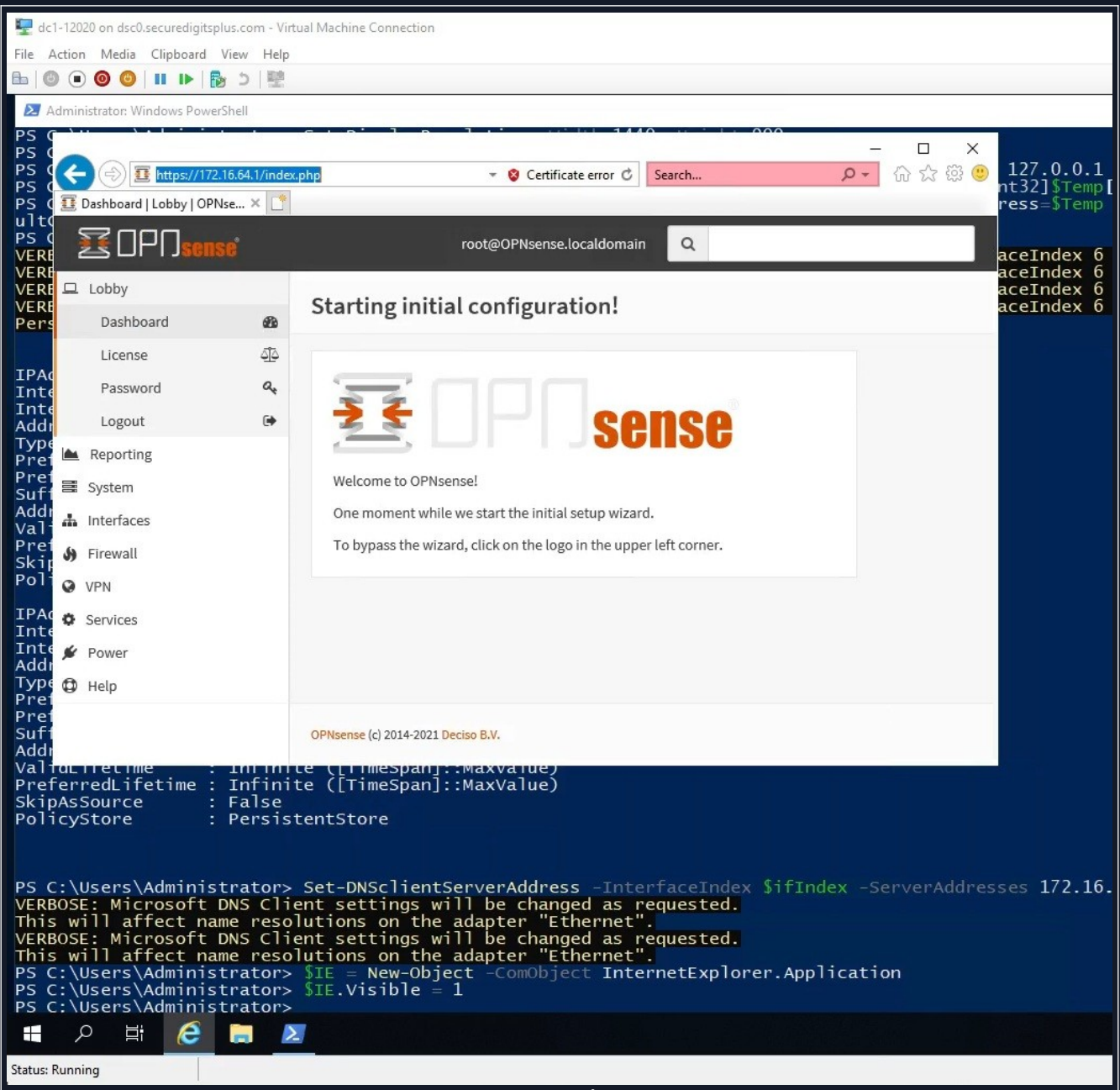
Here, we see that the address I specified, is offering a secure site with a disclaimer. This would not be accessible from the WAN, only from the LAN.



entry-28.jpg

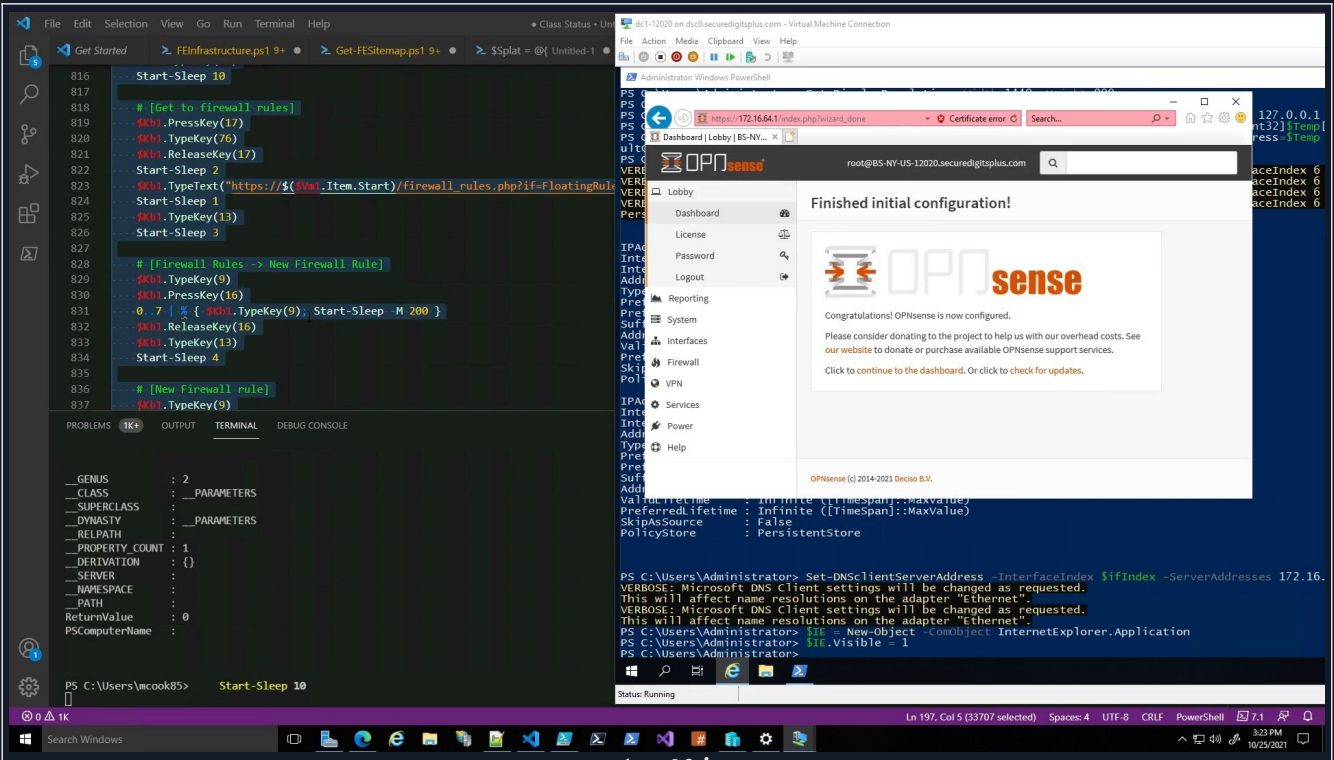
This is the <OPNsense login panel>, available over whatever port it allows, meaning it is configurable. This iteration of FightingEntropy uses the antiquated method of transmitting the credential information, which is not exactly secure when thrown into race conditions.

The work I later did with TCP Session either eliminates or drastically mitigates that, actually.



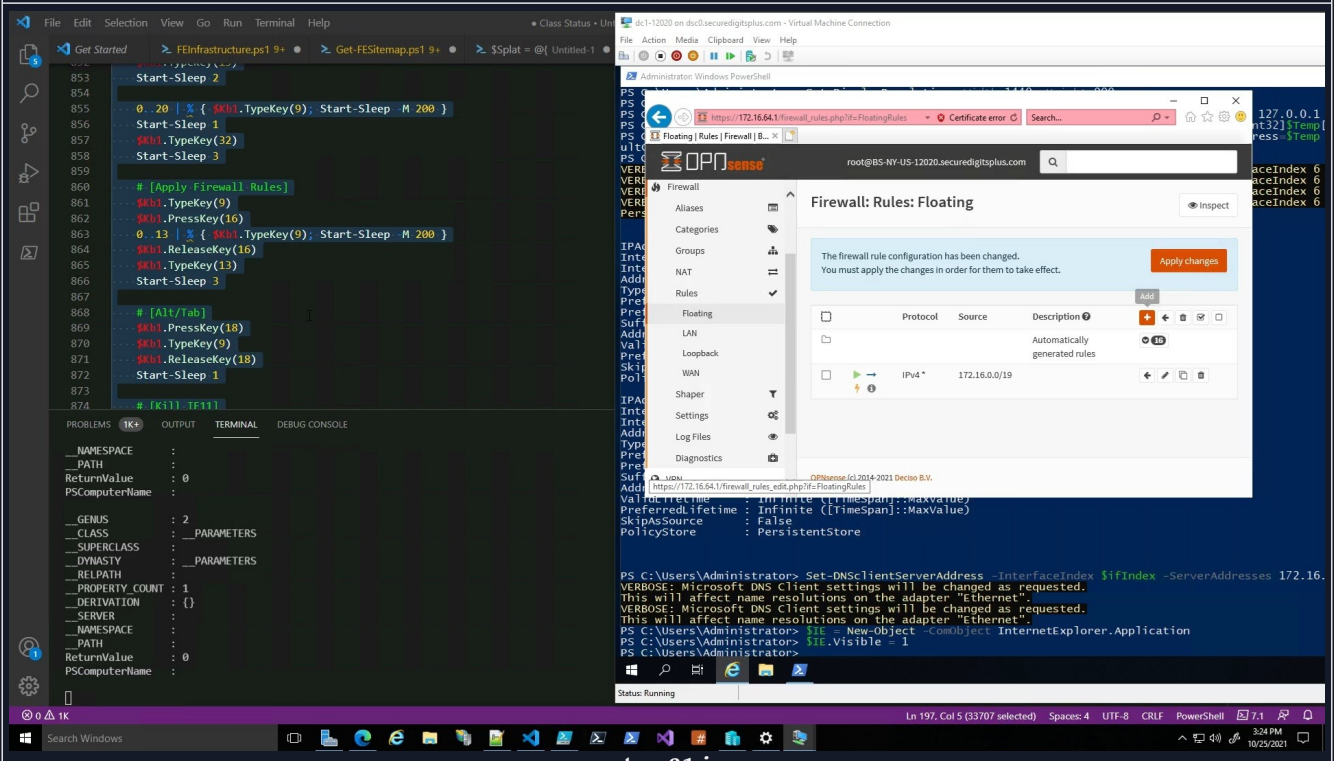
entry-29.jpg

Now we decide to run the initial setup wizard. This is pretty easy, but we wanted to demonstrate interaction with the user interface. We could have staged this from a configuration from the get-go, and maybe set up a SMB share for the gateway to access. Either way, the configuration can be completed in any of these ways, so long as the share or site is serving traffic and the credentials are valid.



entry-30.jpg

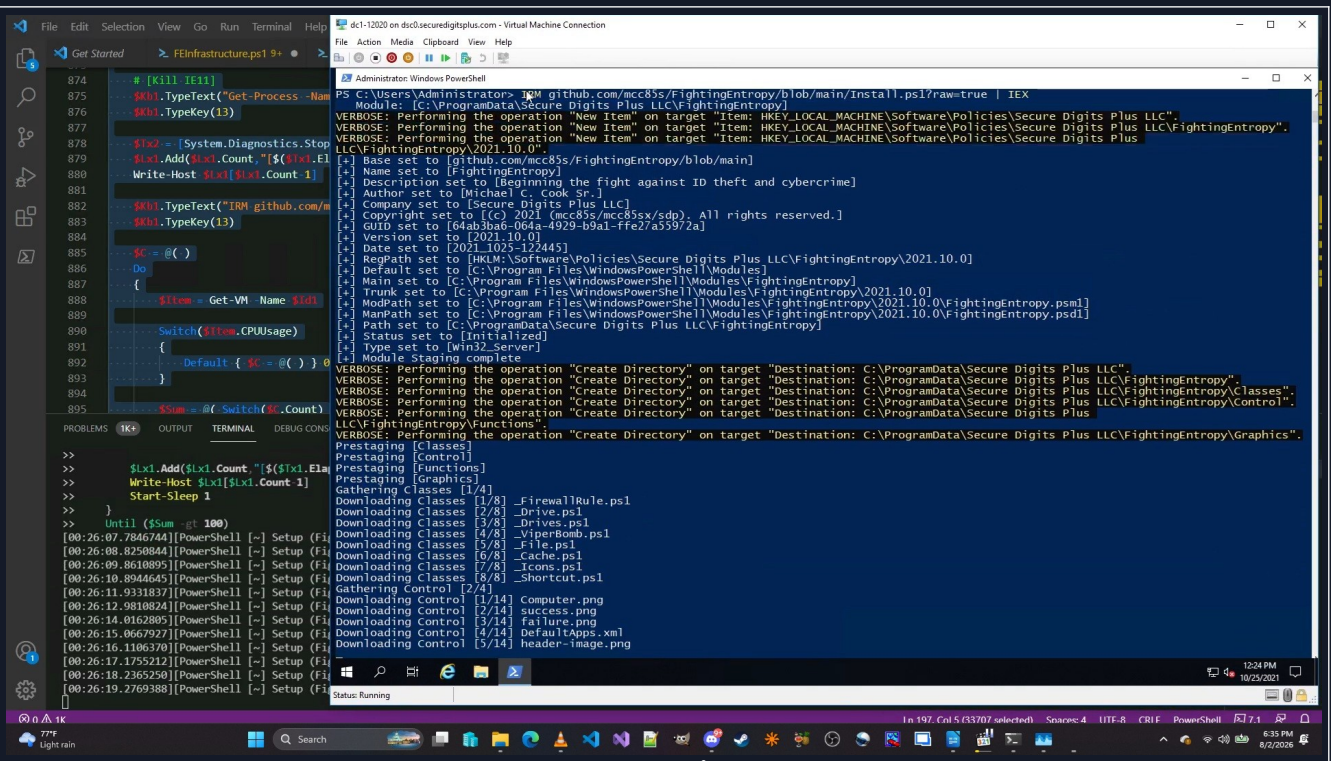
After the initial configuration wizard, additional steps can be carried out. But taking screenshots of ALL these steps would be pretty time consuming, when the video already exists.



entry-31.jpg

It successfully sets a floating rule so that stuff can be downloaded from the internet, and certain PING related stuff works.

[3L]: [FightingEntropy(π)][2021.10.0]



entry-32.jpg

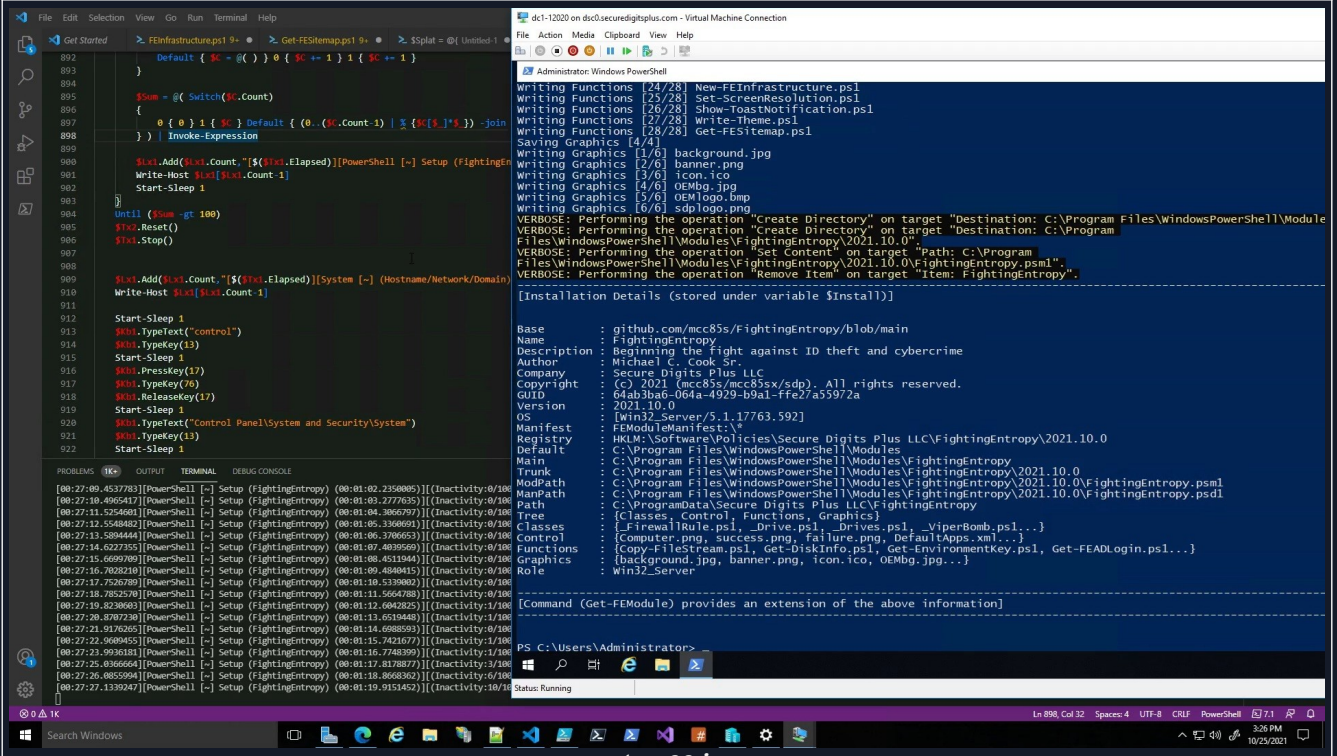
Here, we can see that the 2021 version of [FightingEntropy(π)][2021.10.0] is being installed.

This method was much slower than the full archive version I later implemented, which I was against doing at first because of how often I would update individual files. However, I figured out a way to get both methods to work, and I intend on implementing the same strategy in the next version.

So, if I update the module manifest, it will check the component of the module referencing a particular file.

If the file validates, then it does not update anything nor recompile anything.
If a file FAILS validation, then the current version from the site is then downloaded, it overwrites the existing file, and then recompiles the module.

That's what I was doing with [FightingEntropy(π)][2024.1.0] and I think the (2) prior versions from 2023.



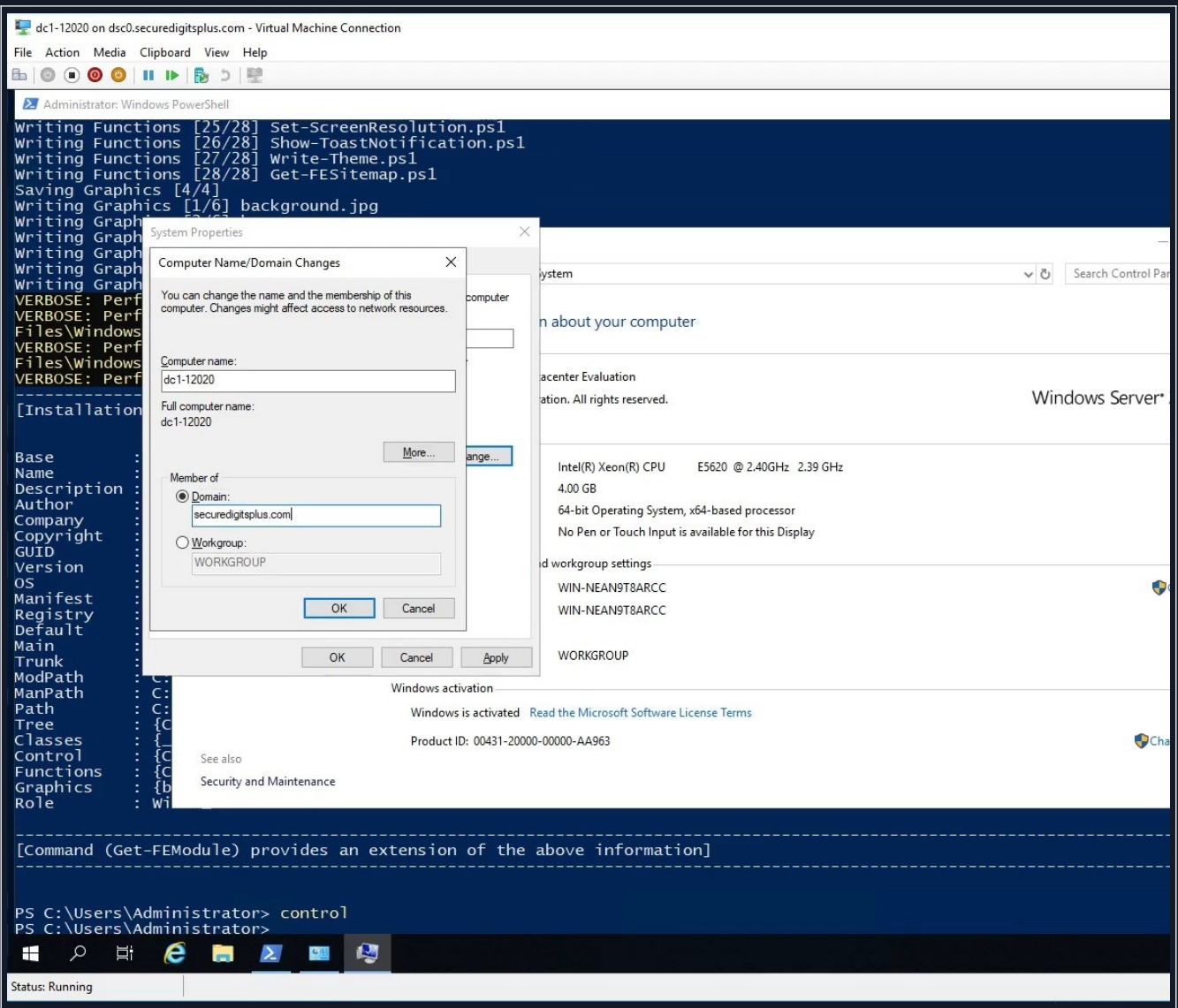
entry-33.jpg

Here, you can see the [Installation Details (stored under variable $Install)]

That is all the module information, except that is the older method that I've made numerous changes to. For instance, many of those values have been replaced by controllers or objects. And, some of those properties seen are just strings. The new version is using objects and controllers with nested properties and this thing is a lot faster than any of the PowerShell based versions of the module.

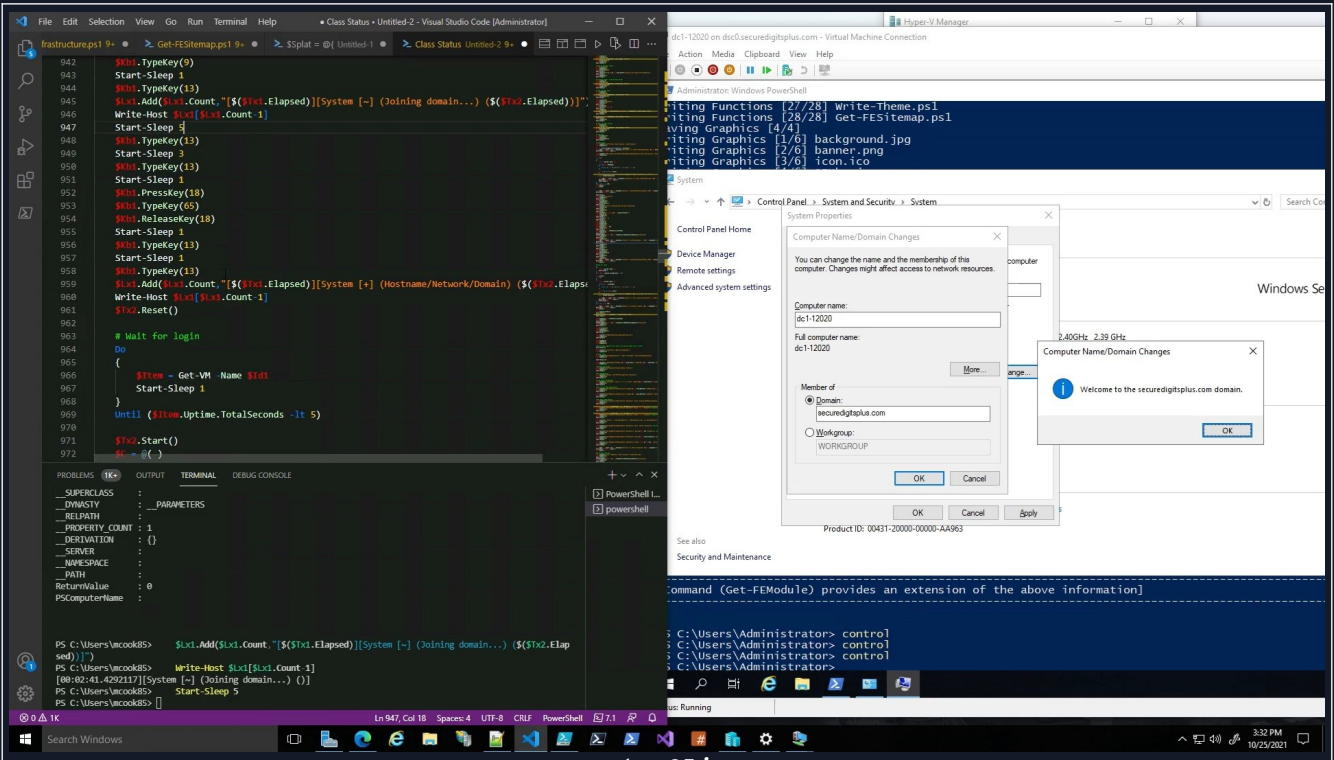
Mainly because C# is just the way to go with a lot of the work I was doing in PowerShell.
Problem is, it requires a LOT of forward thinking in order to support Linux.

[3M]: Server Configuration (Phase 2)



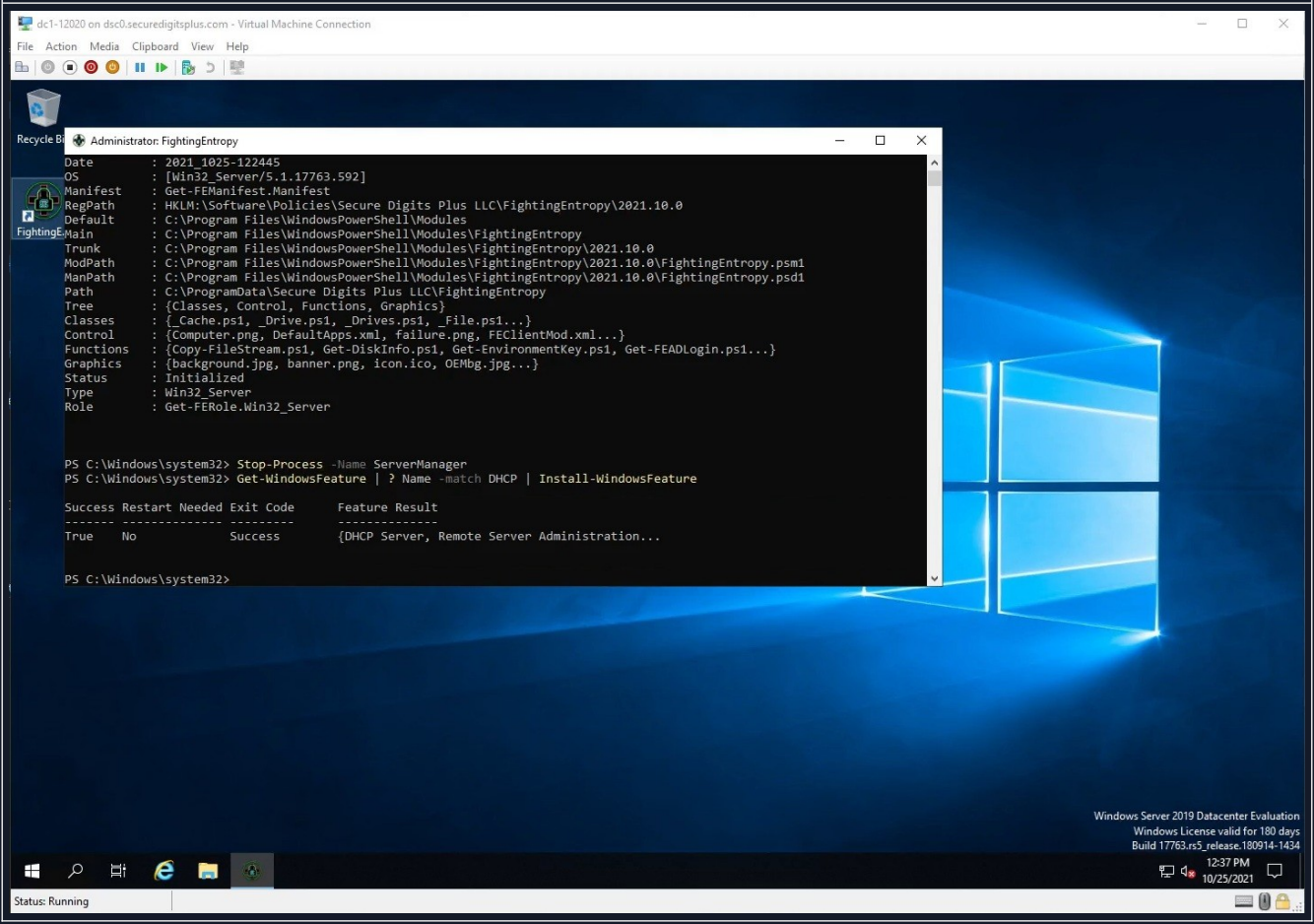
entry-34.jpg`

Here, we see that I access the system properties from the control panel, but these various DLL's are accessible via P/Invoke, which is something I'm working on implementing.



entry-35.jpg

So here we have the final result, I had to stop and troubleshoot during the demonstration, but that just goes to show my ability to do so while recording, and to keep the thing rolling.

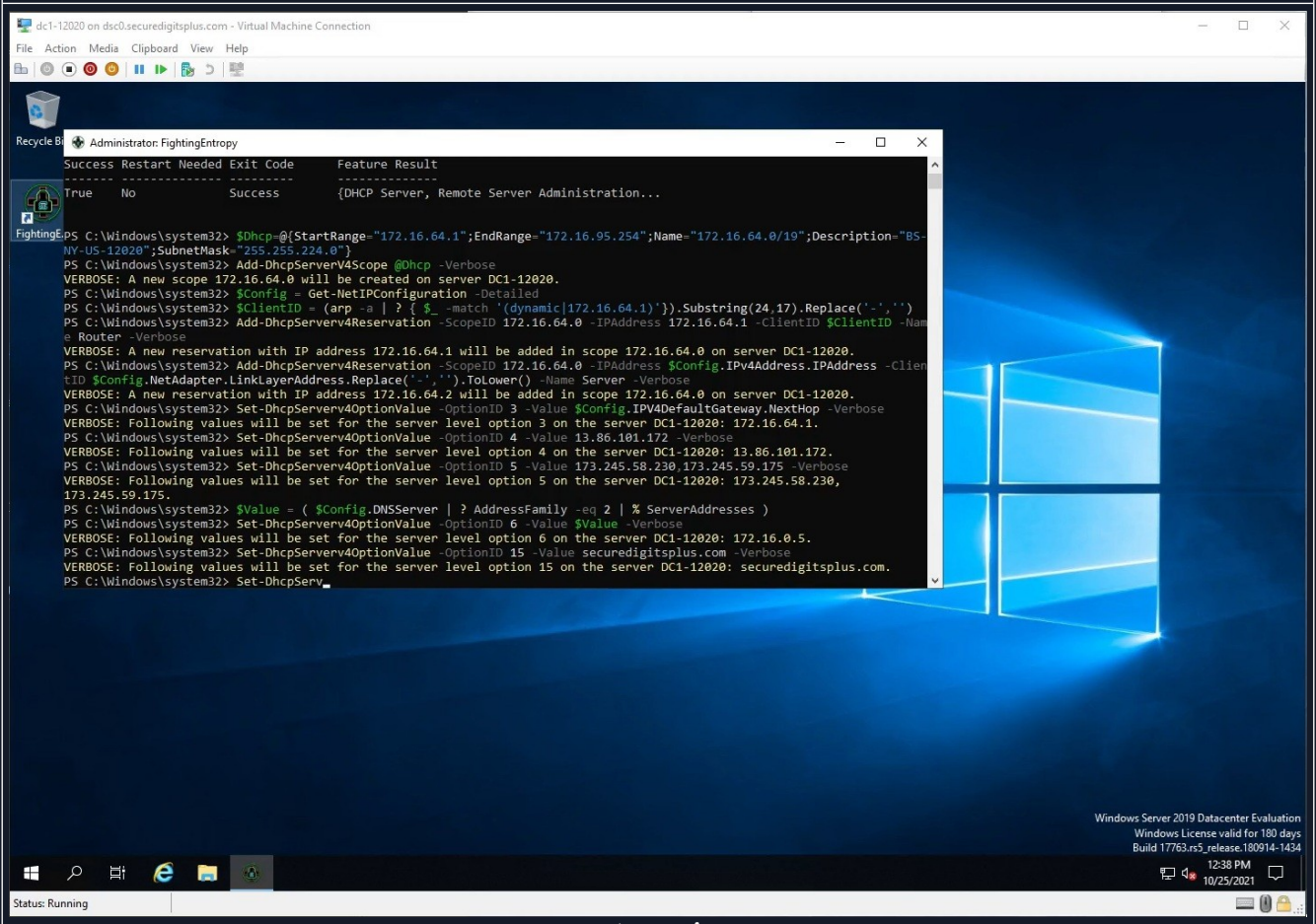


entry-36.jpg

So we see here, that the module (shortcut/icon) on the desktop opens up this PowerShell console, to which it provides elevation and manages the prerequisites for Get-FEDCPromo. DC Promo is a utility meant to onboard a Windows Server instance to become an Active Directory Domain Services controller, which requires some components of Active Directory to already be installed.

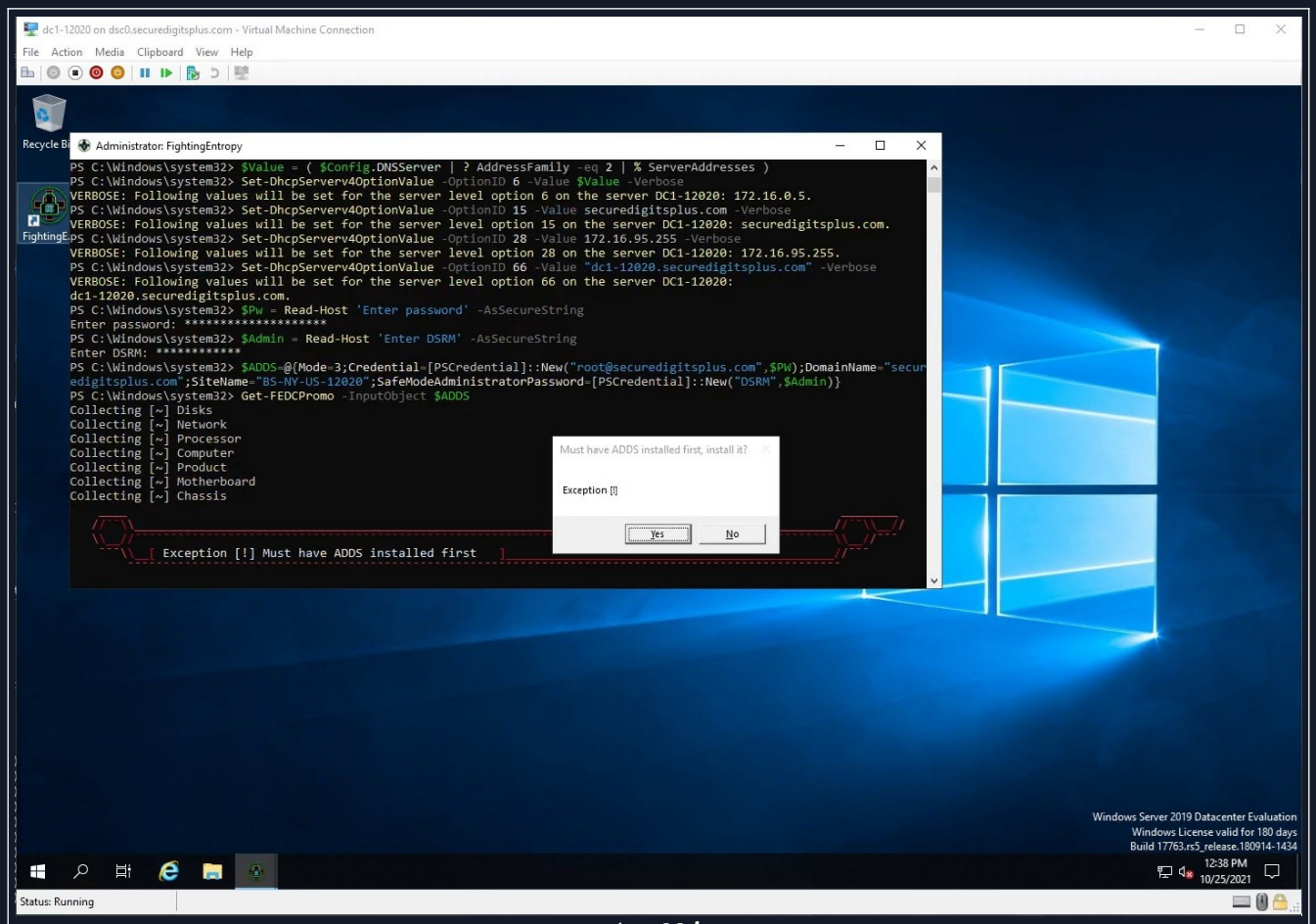
However, I have a way of automating that.

[3N]: Get-FEDCPromo (Phase 1)



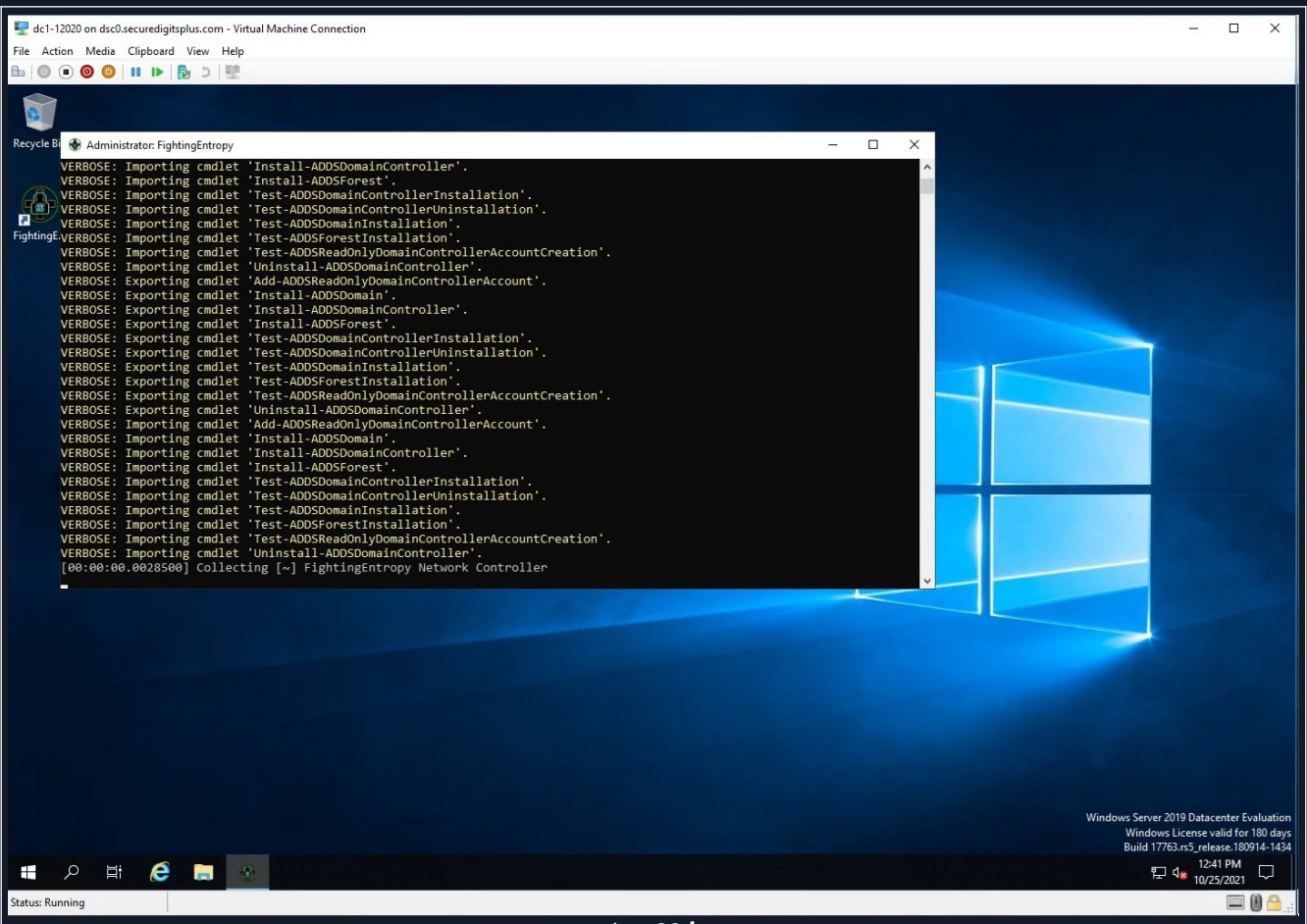
entry-37.jpg

You can see very clearly that I know my way in and out of DHCP scopes and configurations, which some of these need to be set up before I start upgrading the thing to a domain controller. You don't NEED DHCP on the server, but I think it is best to deploy it alongside the domain controller promotion, because then you can have Active Directory maintain strict control over those subnets. Or at least to some extent.



entry-38.jpg

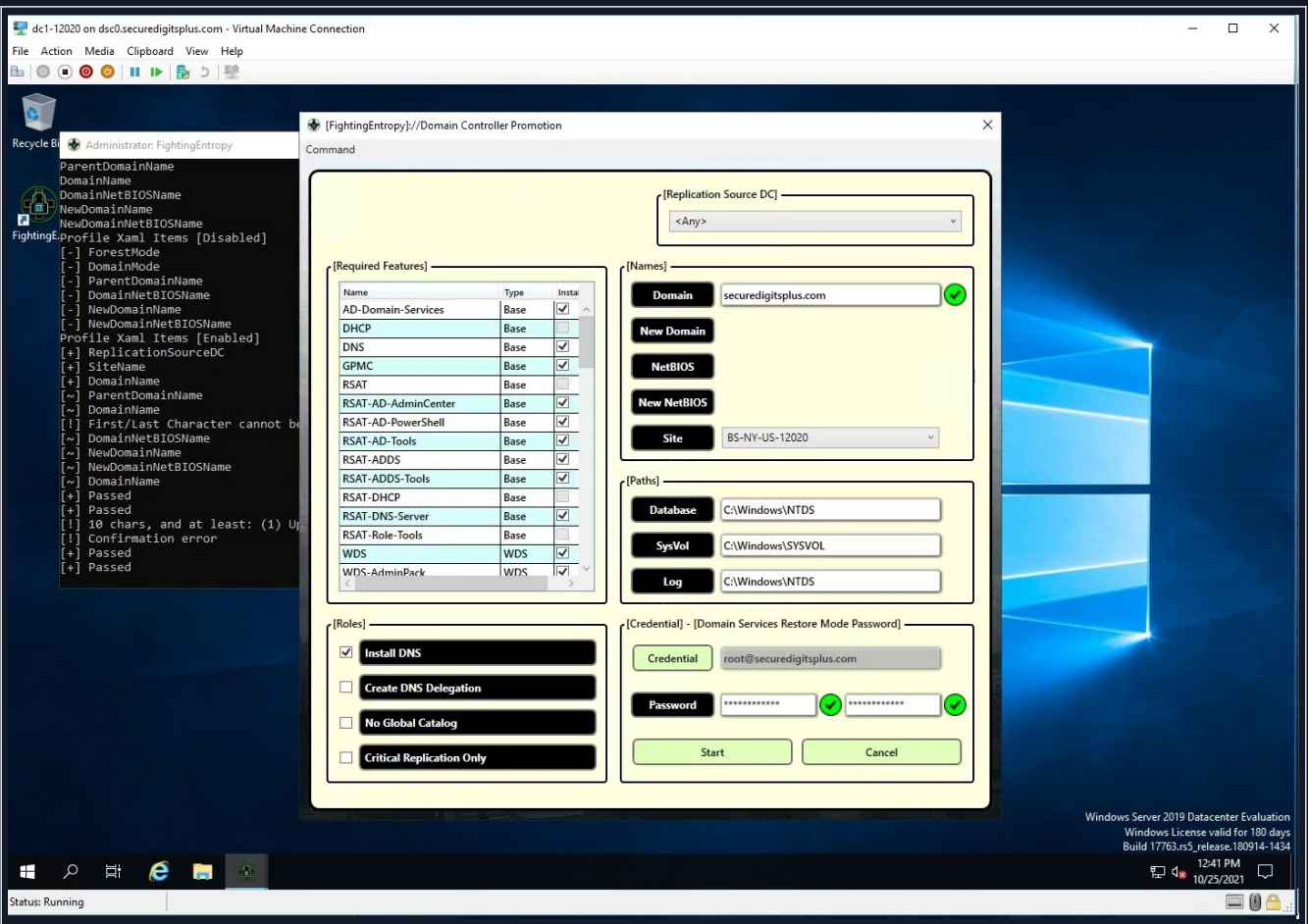
So, when the information is sent to the current instance of PowerShell, it can stage all of these things correctly. Right now, the exception “Must have ADDS installed first” appears. That will install the ADDS feature automatically, and then proceed to launch the FEDCPromo utility.



entry-39.jpg

Once Adds is installed, it can then run the function that was staged beforehand. I have a GUI that goes along with this, but I was testing my ability to transmit the "seed" instructions to test Get-FEDCPromo, which requires a successful login to Active Directory to work, but this was staging a new primary domain controller.

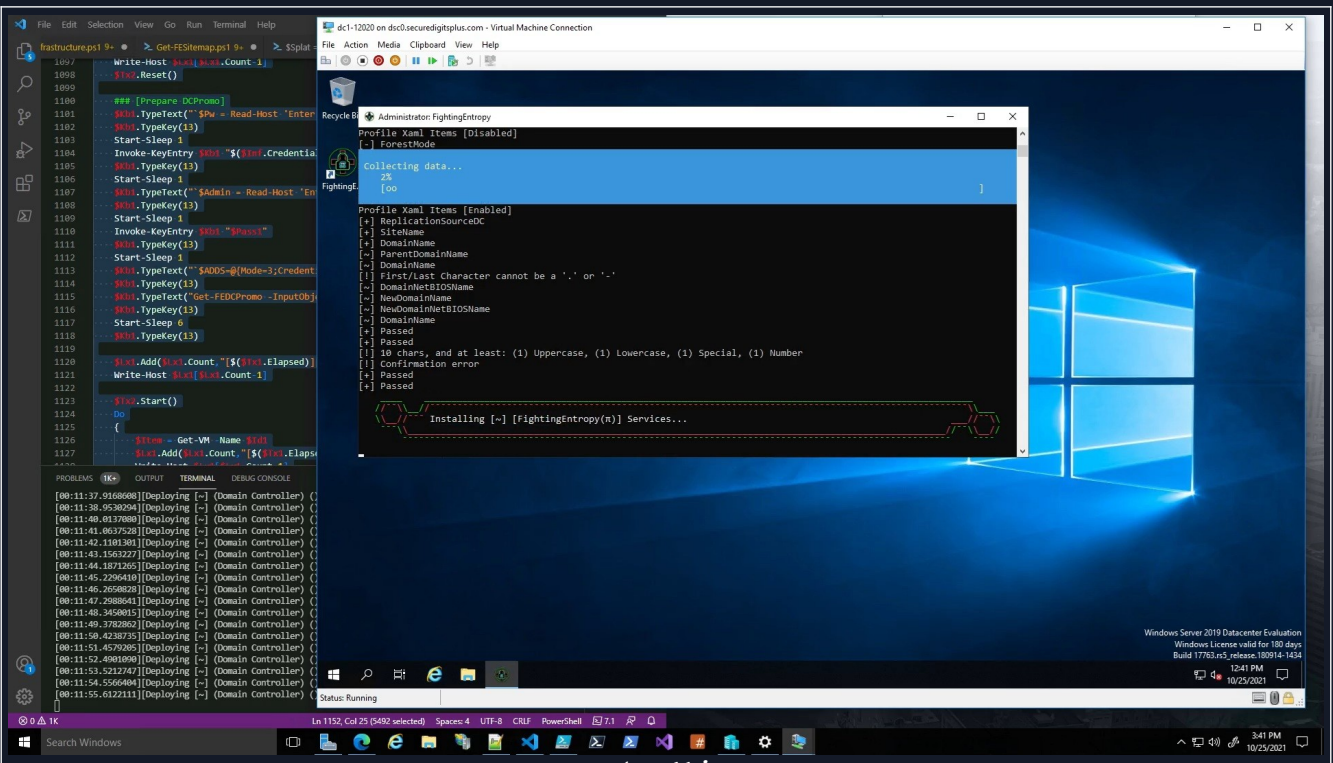
[30]: Get-FEDCPromo (GUI)



entry-40.jpg

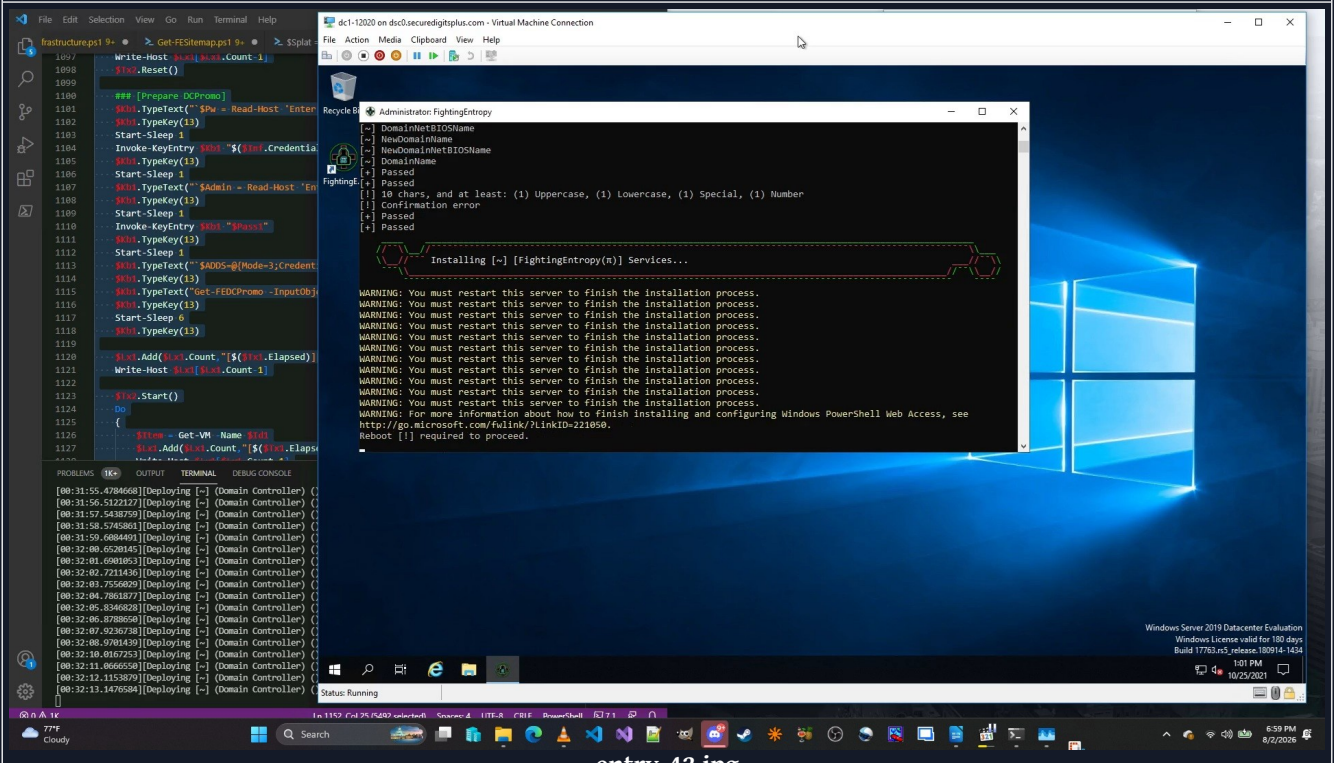
This is what the user interface looked like. I've made numerous style changes to my overall GUI's, but I had a lot on the screen here as can be seen. The groupbox items would wind up being eliminated, some of the checkboxes can be replaced by a datagrid, bunch of ideas on how to make this more compact while still having the same amount of information accessible.

When the device is ready to install a bunch of features via that InputObject parameter...? The system will procedurally install the necessary features, and then do a reboot and continue the process that was not completed after rebooting. This means you run this utility once, and as long as everything checks out, it will 100% upgrade it to a legitimate domain controller.



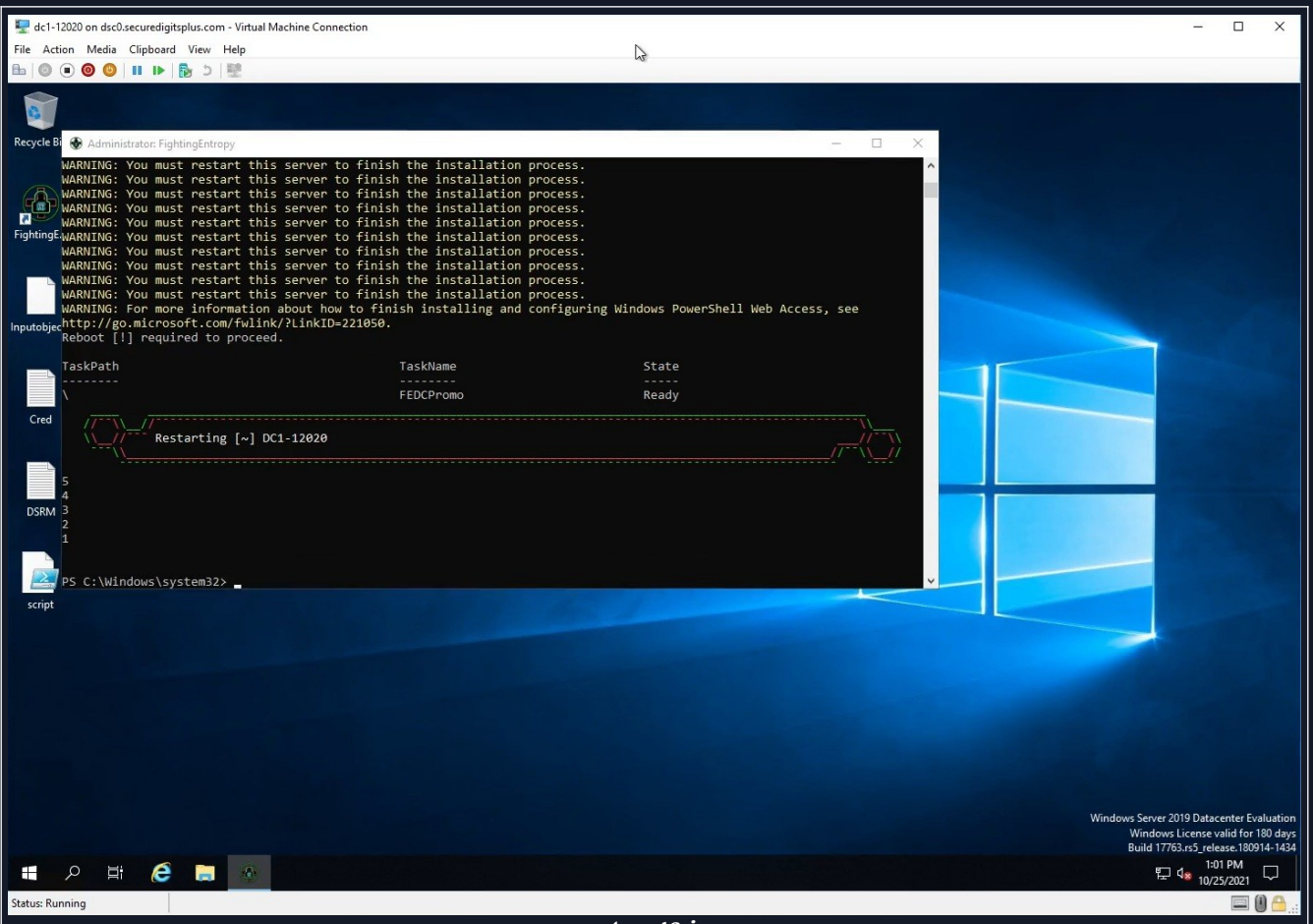
entry-41.jpg

This is what the console will look like after it has kicked over and initialized. The timer on the console says ~11m 50s, and this process takes a while and like I said, does require a reboot at some point. This whole process requires a few reboots, but I think I can get that number down somehow.



entry-42.jpg

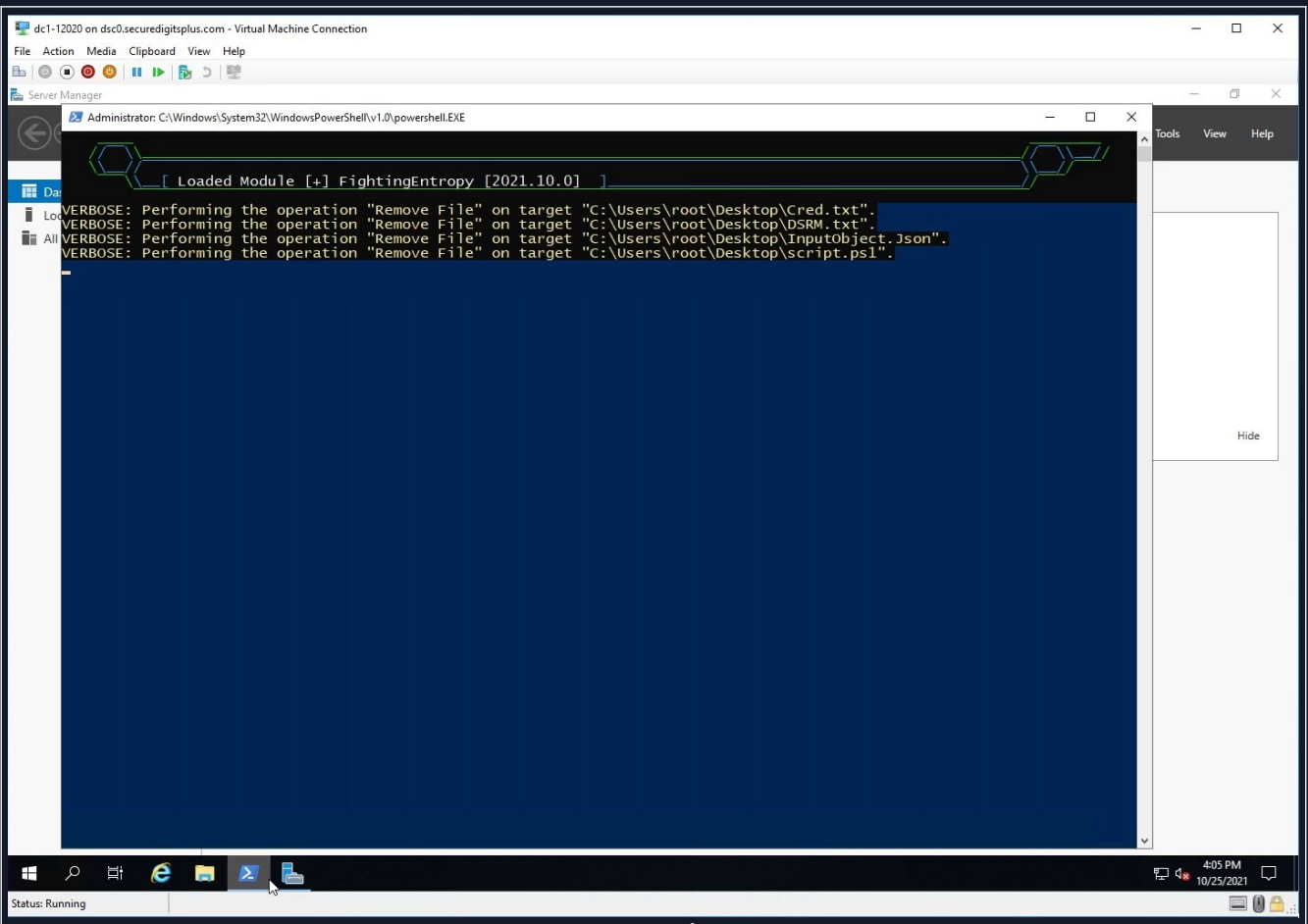
So, this process took about 20 minutes, and that version was not displaying the features it was installing. I've since fixed that. Regardless, this successfully creates a task in the scheduler after reboot, where it can continue from where it left off and finish the promotion.



entry-43.jpg

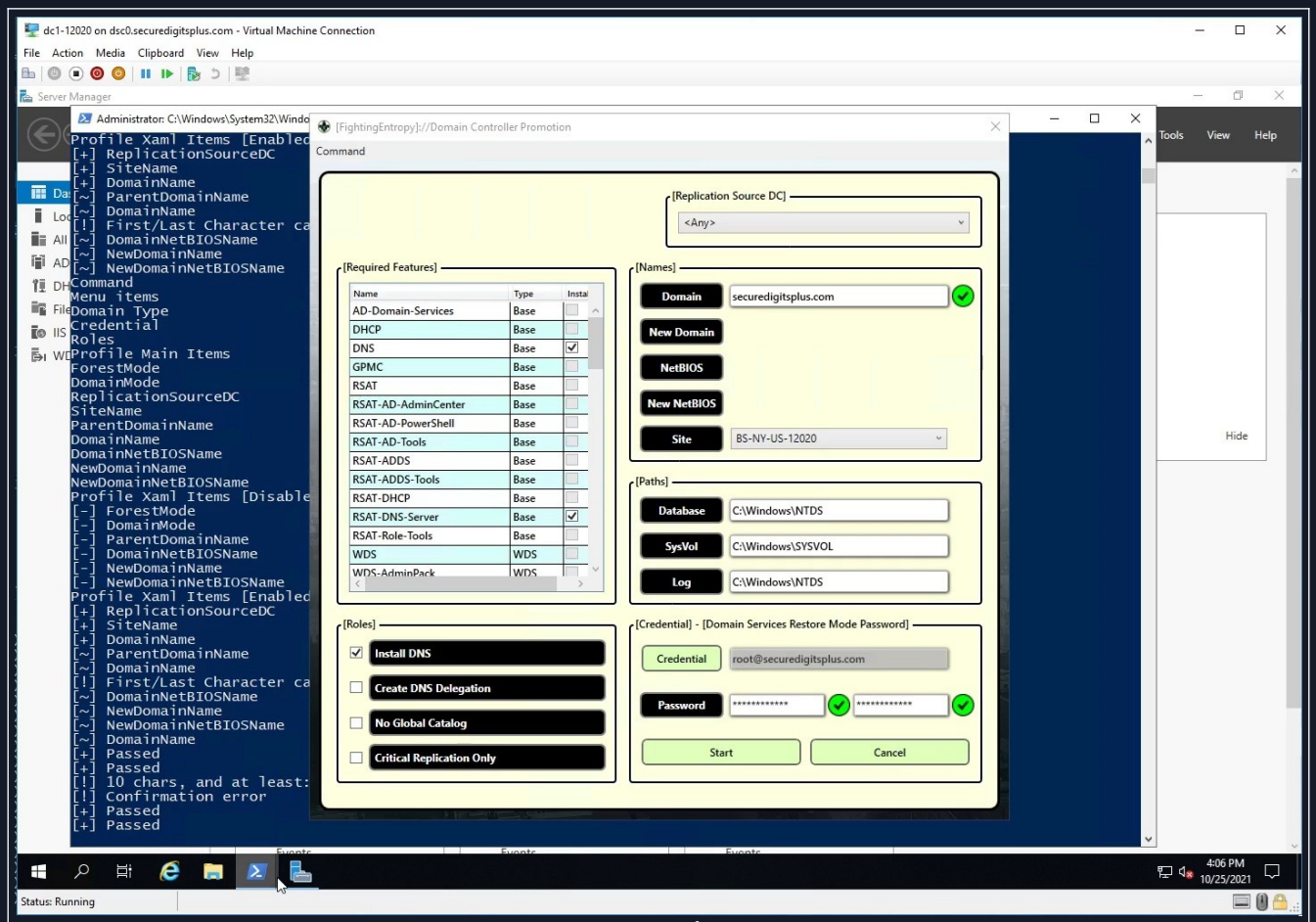
Notice the items on the desktop? There's a better option than that.

[30]: Get-FEDCPromo (Phase 2)



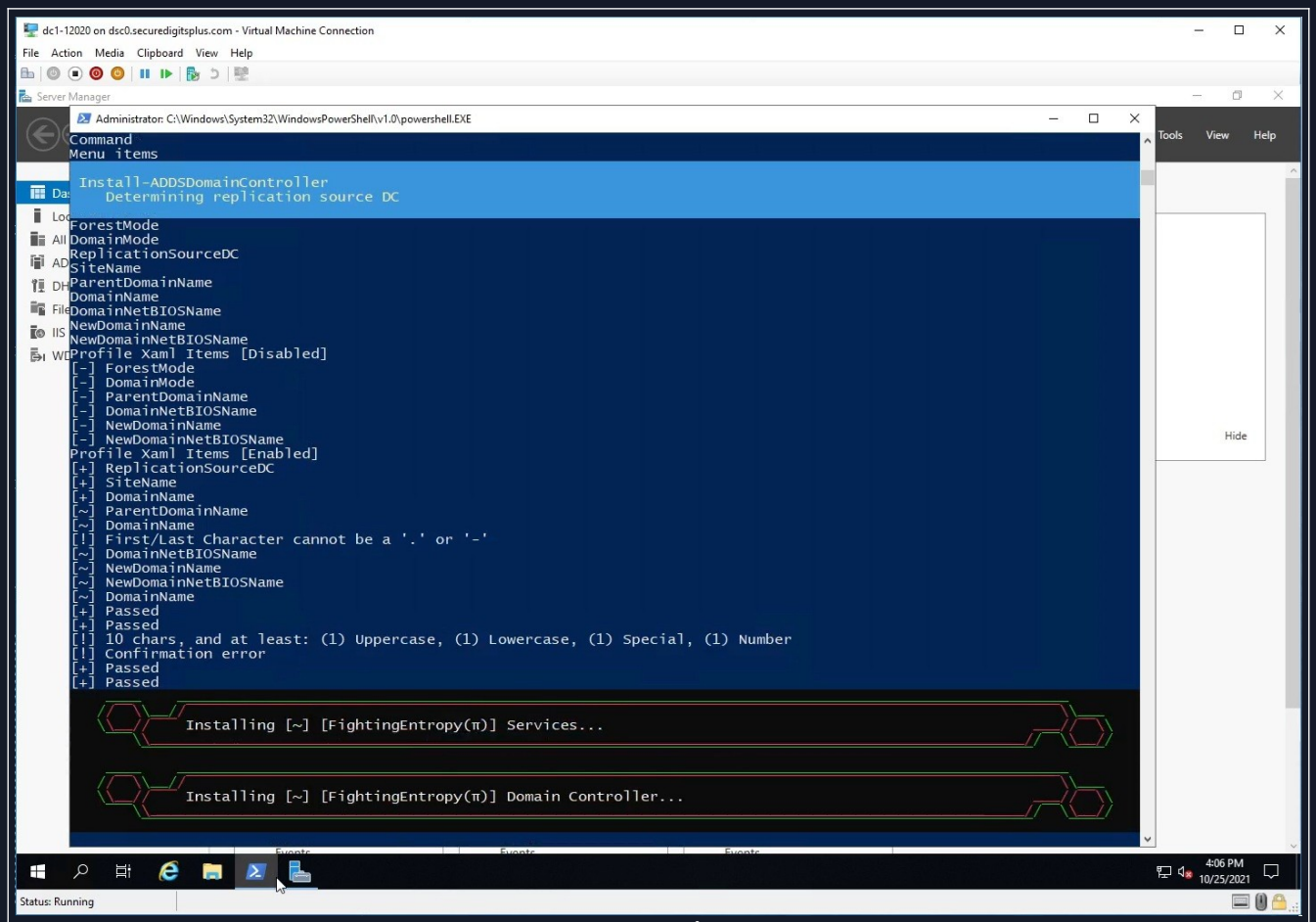
entry-44.jpg

After the reboot, those files are removed, but they are absorbed by the post-reboot process. This is similar to what Desired State Configuration does.



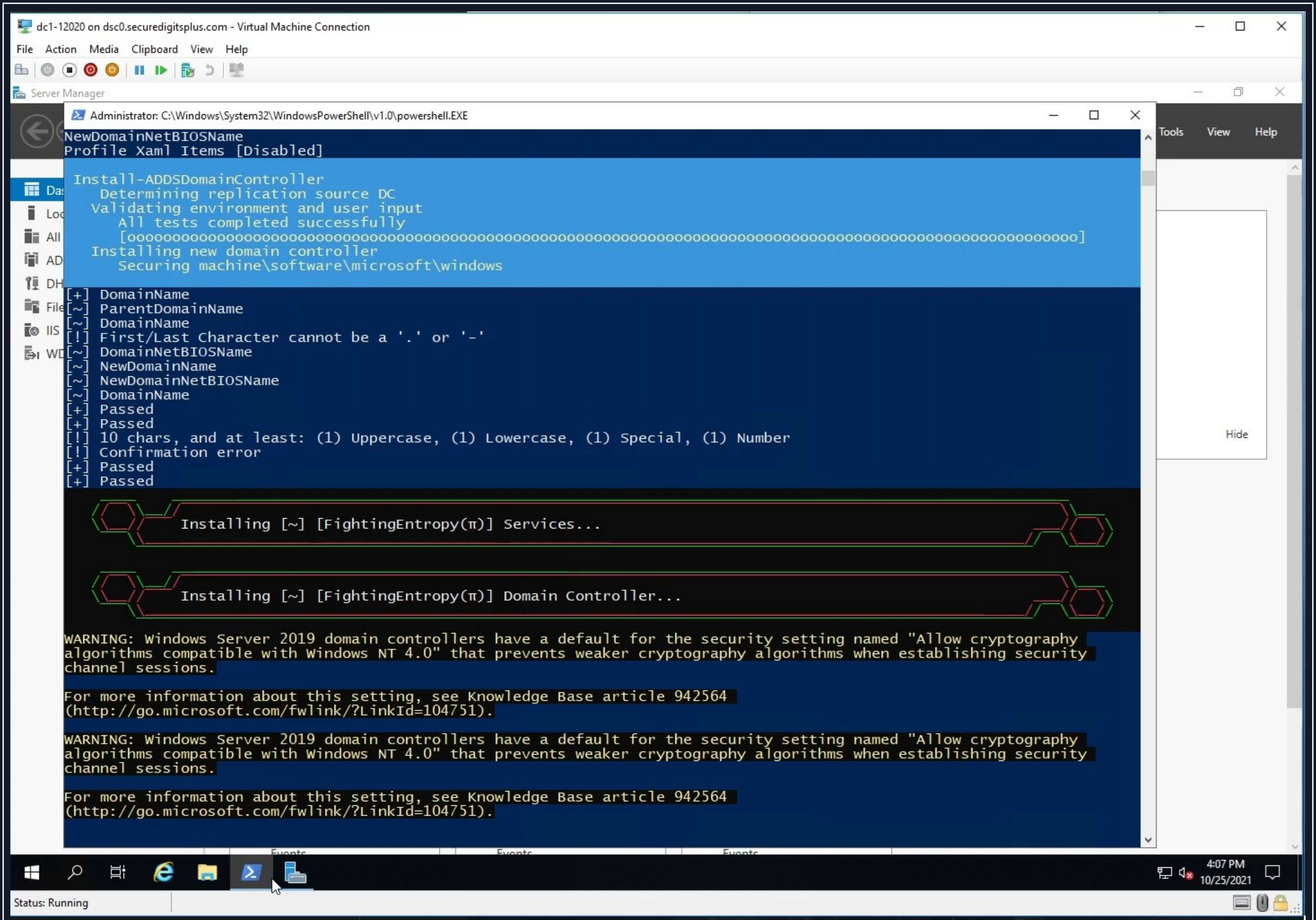
entry-45.jpg

It does a second pass with the utility, and finishes the promotion process. It is best to let ADDS install the DNS services, as it will configure everything for ADDS and DNS simulaneously.



entry-46.jpg

It successfully connects with the account it was provided.

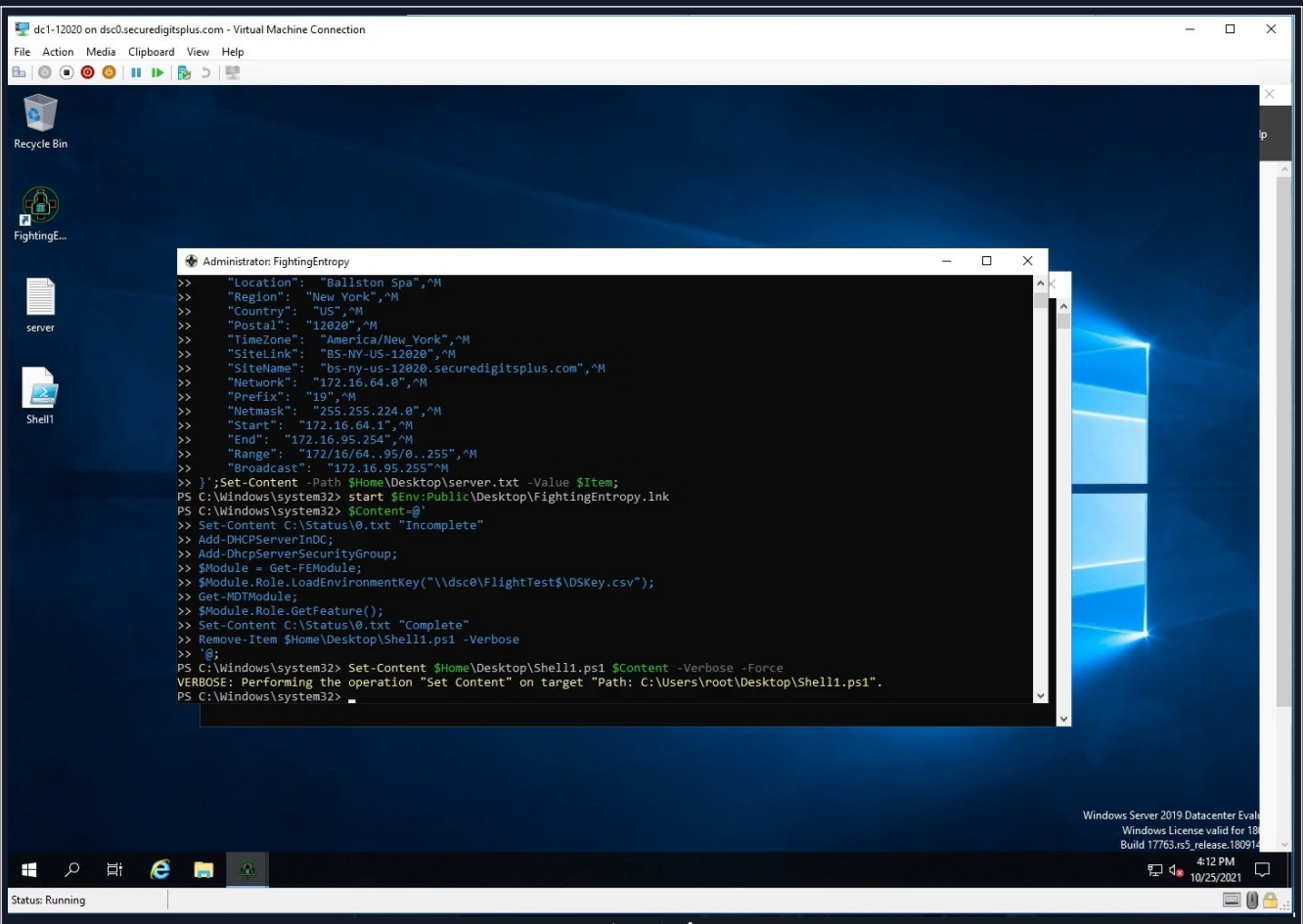


entry-47.jpg

It passes all tests successfully, and installs ADDS.

The system winds up rebooting and then when it reboots, it is now a domain controller, and the ADDS credentials and object can be generated and created.

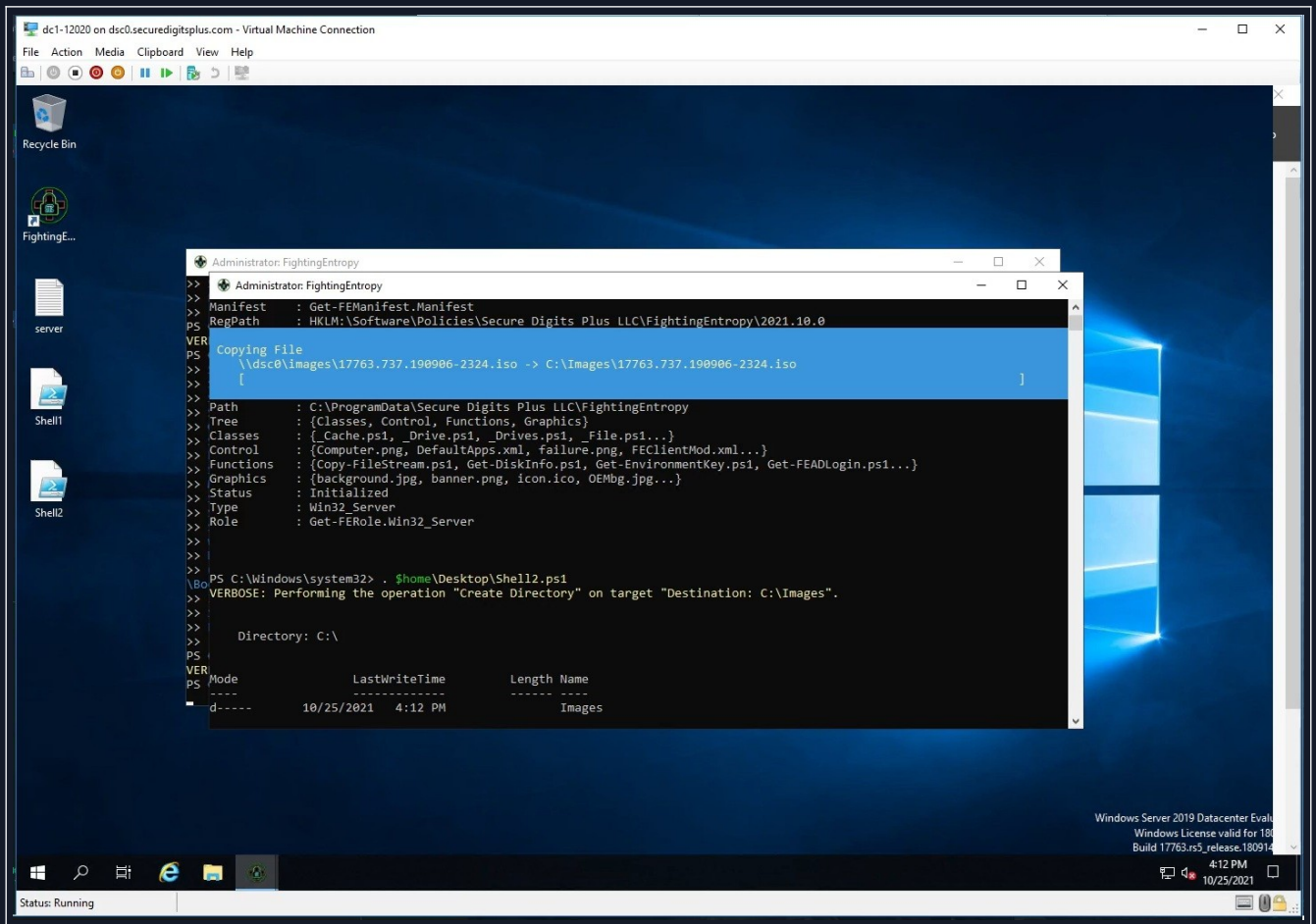
[3Q]: Server Configuration (Phase 3)



entry-48.jpg

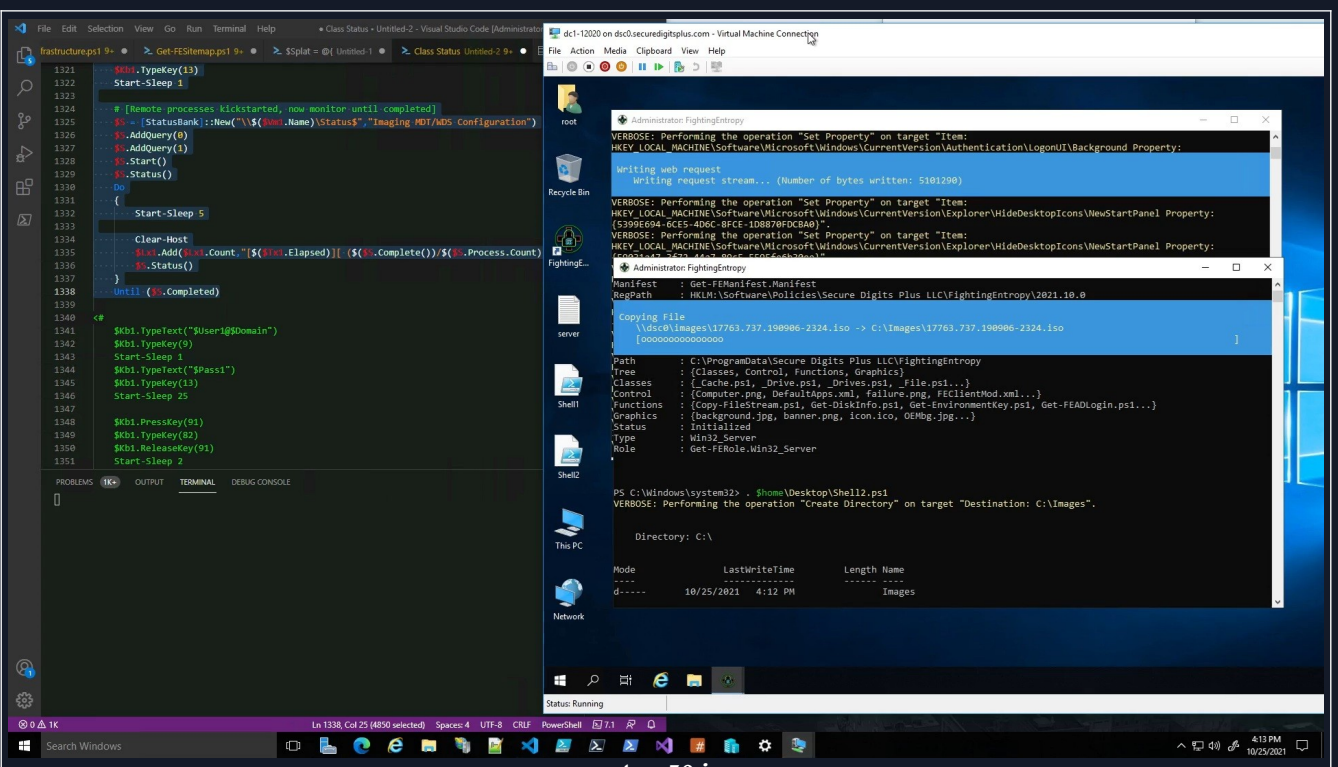
In this phase, we wind up installing (MDT/WinADK/WinPE), as well as transferring (DVD.iso) files, and staging WDS for a deployment. This video doesn't cover all of that, but it shows the process of preparing those environments.

The old configuration method passed the values over the keyboard, not a great idea.



entry-49.jpg

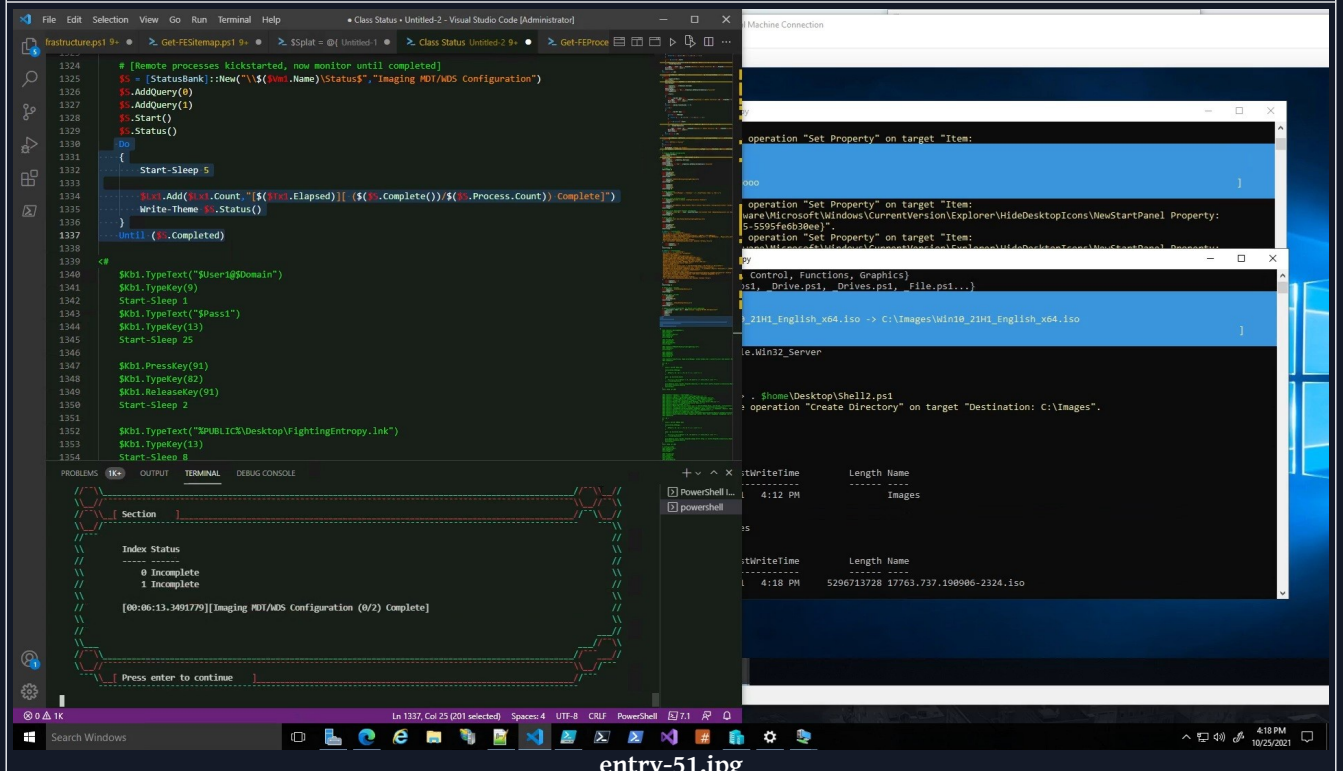
This process here created (2) different scripts that could easily be managed by the module itself now, without having to write stuff to file, it just stays in memory and can log stuff. Back then, I hadn't figured out the TCP Session controls, but that would effectively replace all of this script writing and execution.



entry-50.jpg

So here we have the (2) scripts processing in parallel. Which is fine, because one script is installing 1) MDT, 2) WinADK, and 3) WinPE, while the other is simply transferring the 17763.737.190906.2324.iso image over from an SMB share utilizing the custom file copy function I wrote.

All of this interaction is avoided by having the <module controller> handle a <transmitted> series of <scriptblocks>, which I later implemented. I had <scriptblocks> hardcoded into the <VmControllers> I created, I would eventually rip that out, though some defaults are still part of the class factory.

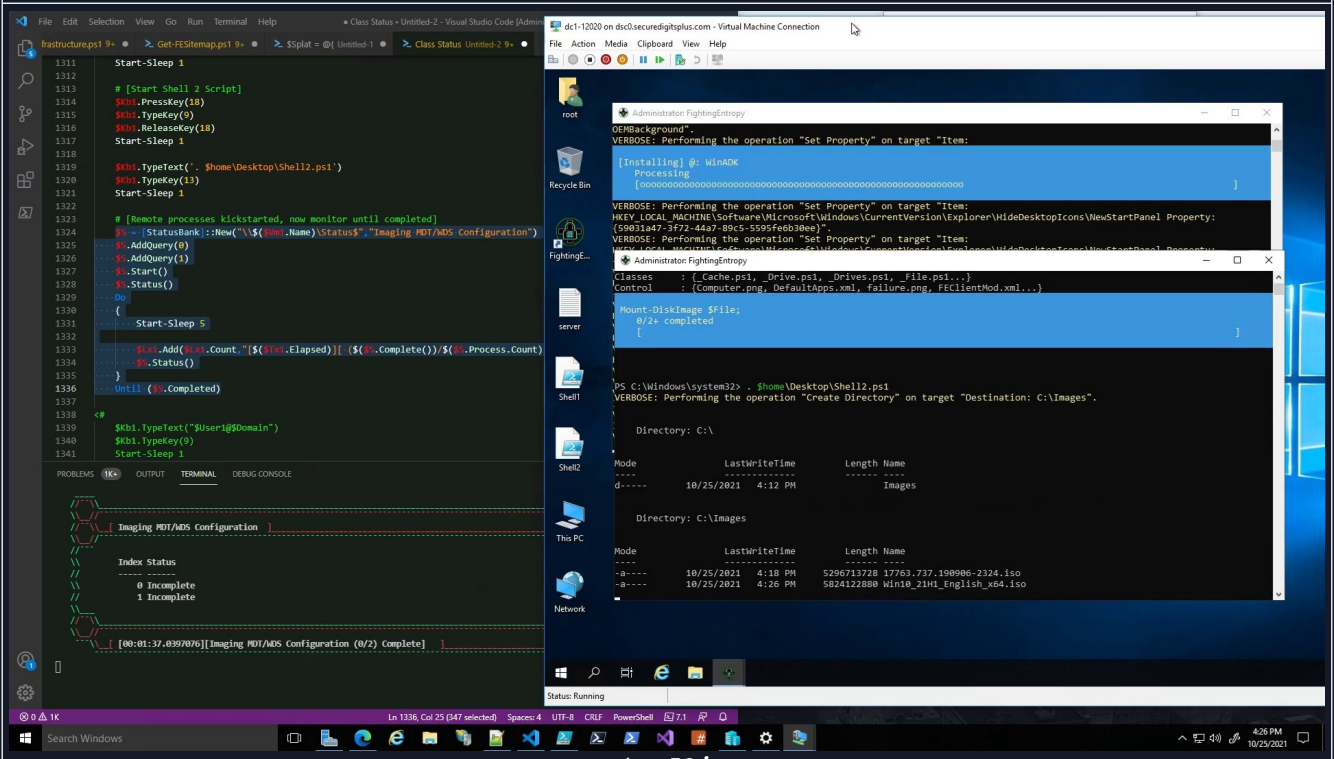


entry-51.jpg

Here, you can see some progress reports making their way back to the <VmController> console. It wasn't the <VmController> console at this point, I was manually writing stuff to the console.

Figuring out how to use the SMB share to exchange progress reports with the host system is a real pain in the neck. But, runspacing via the <Thread.Controller> should alleviate some of that.

Each script is effectively running in its own <runspace>, which can actually be managed by the <Node.Controller> and the <Thread.Controller> (*like I just mentioned*). Normally you would want to use SSH or something to do a lot of this configuration, but there are requirements in order for SSH to work first.

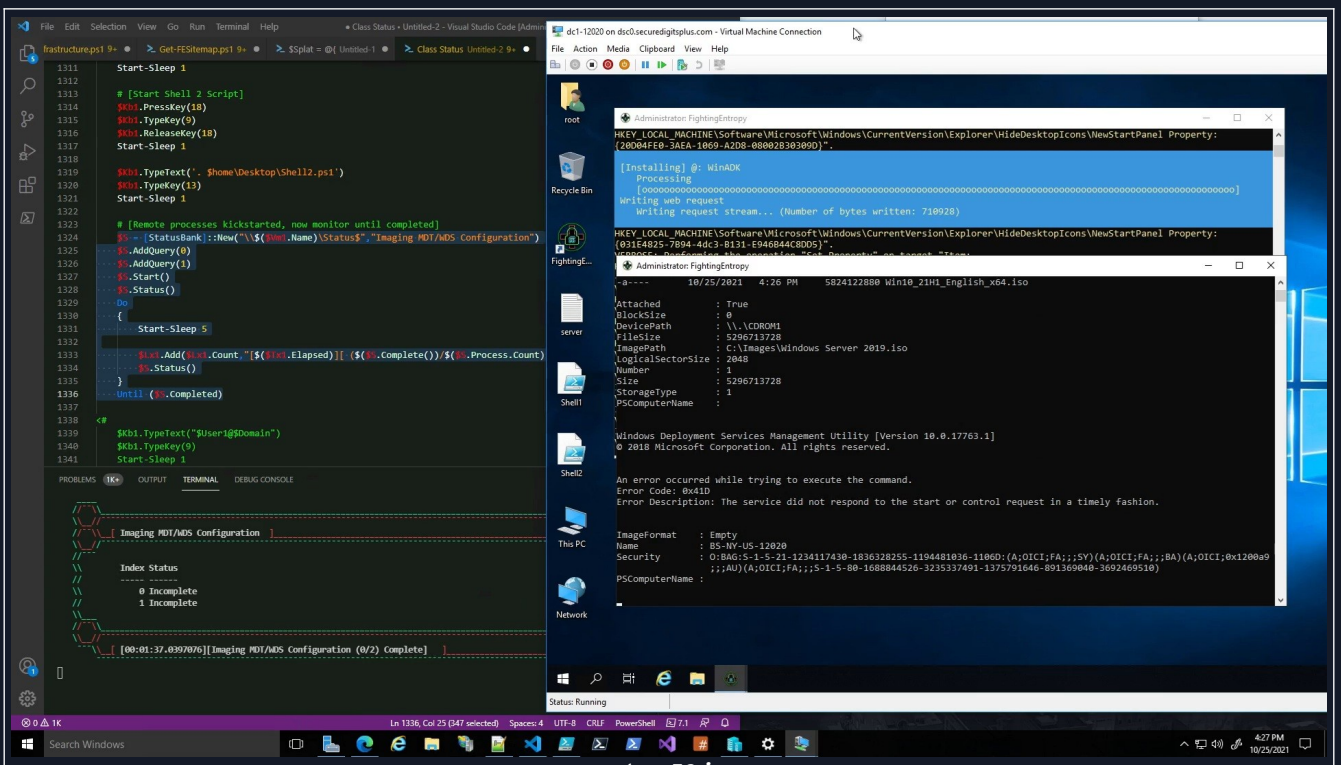


entry-52.jpg

At this point, the image manifest has been downloaded, and the system can begin to inspect the disk images, like a program I used to use called DISM++. Yeah, I wound up offloading a lot of what DISM++ does to the <Image.Controller> in IDS.

The idea at that point was that there was a custom function in the module that would look at an image manifest, and then transfer those files from one end to another, and then when the images were there, find a way to extract the (*.wim|*.esd) files for MDT to create a deployment share with.

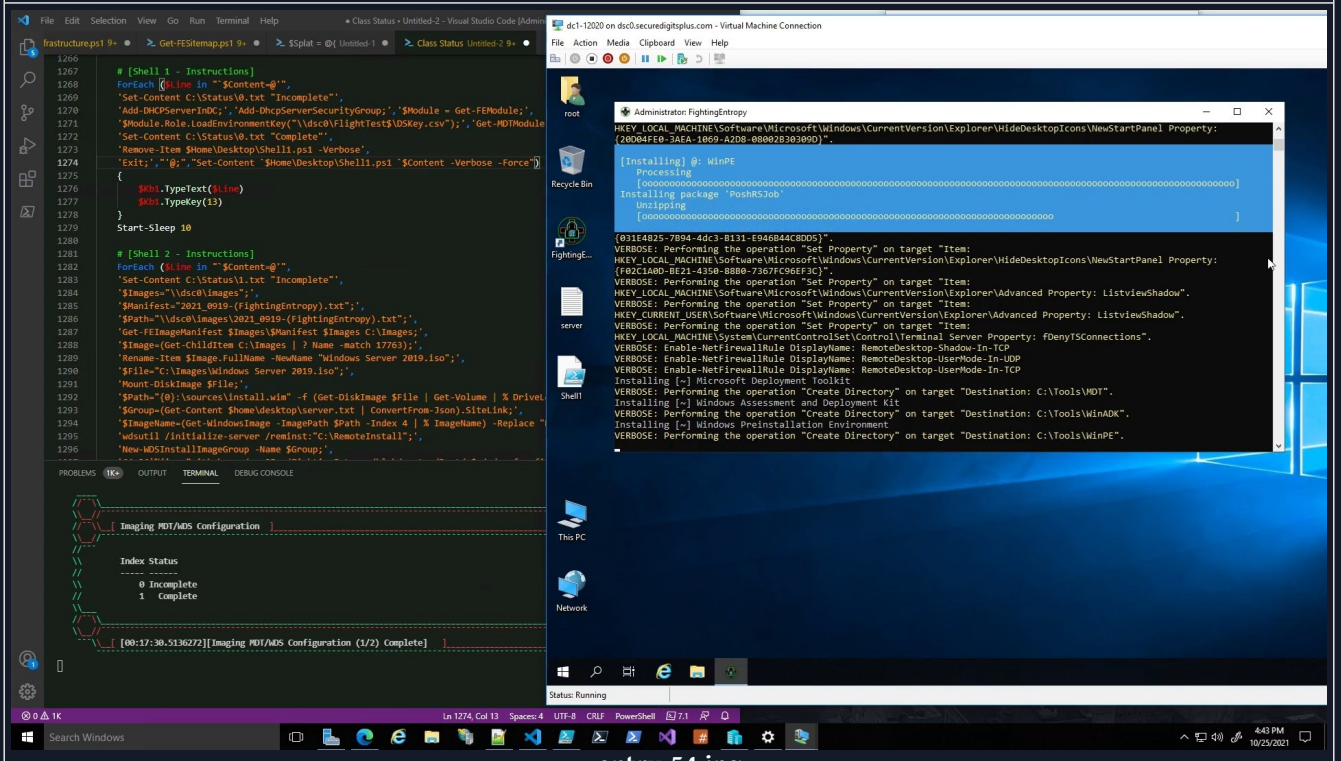
Would've been better to just extract the wim files from the source system, but if you want consistency between the child item computers, you may want to use the original (*.iso)'s as opposed to the embedded (*.wim) files.



entry-53.jpg

At this point, WDS will throw that error because a couple of files are actually missing from the installation by default. I'm not sure why that was the case with Server 2019, but it was.

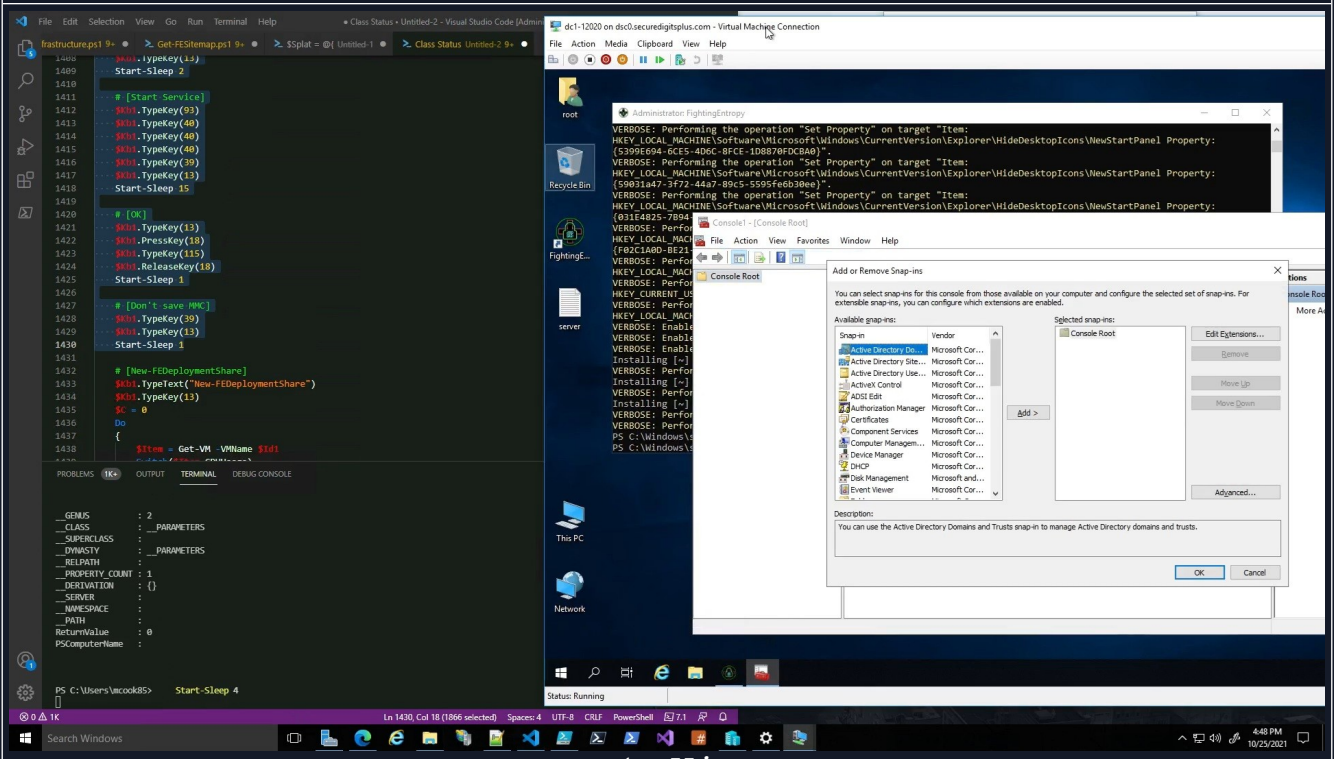
I have other videos that show WDS getting fully deployed, and then successfully restarting. Then, having MDT generate a new deployment share and build boot images. I'll cover that shortly.



entry-54.jpg

Ok, at this point, WinPE is finally installed. I had to continue screwing around with this installation script for

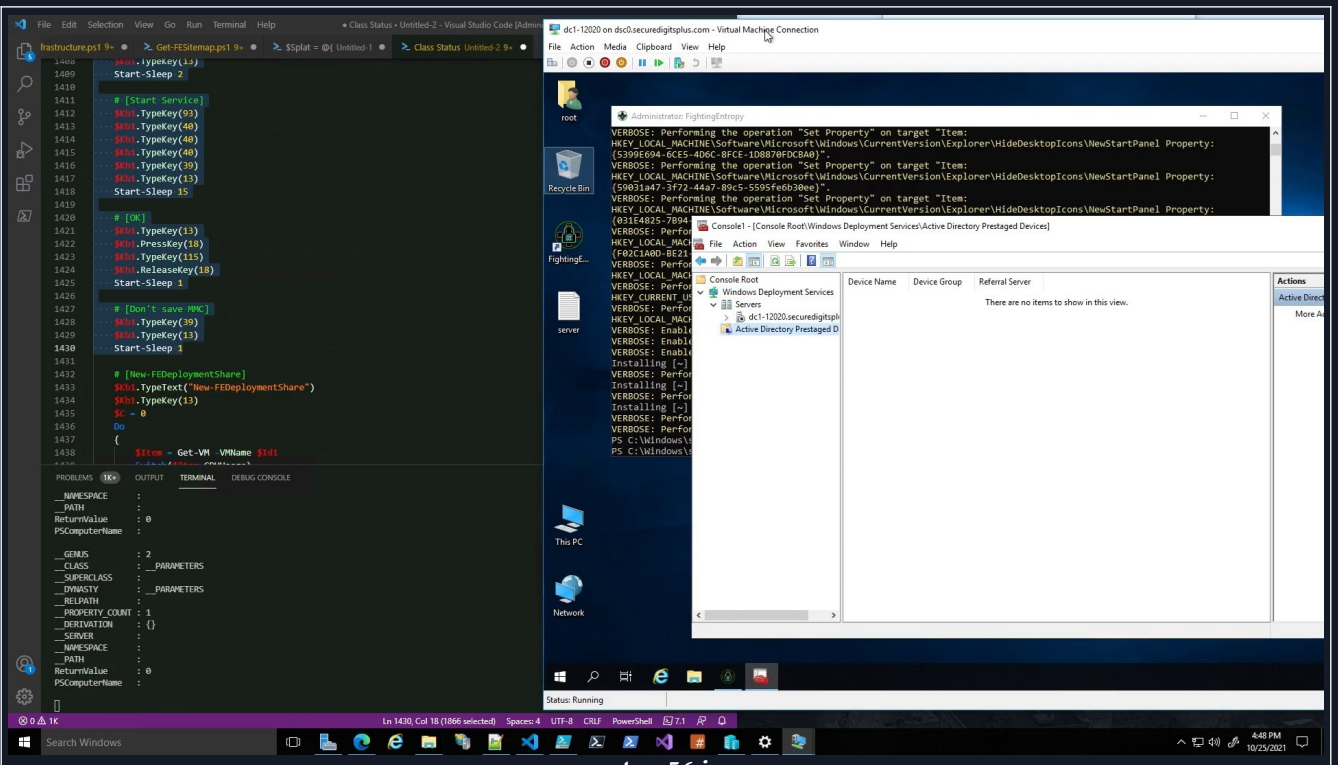
(MDT/ADK/PE) because they run a process that then kickstarts WMI Modules Installer, and then that thing takes a while to download and install, but when it's done it doesn't know it's done, so it just keeps going and it wasted a lot of time time figuring out how to ACTUALLY manage that process.



entry-55.jpg

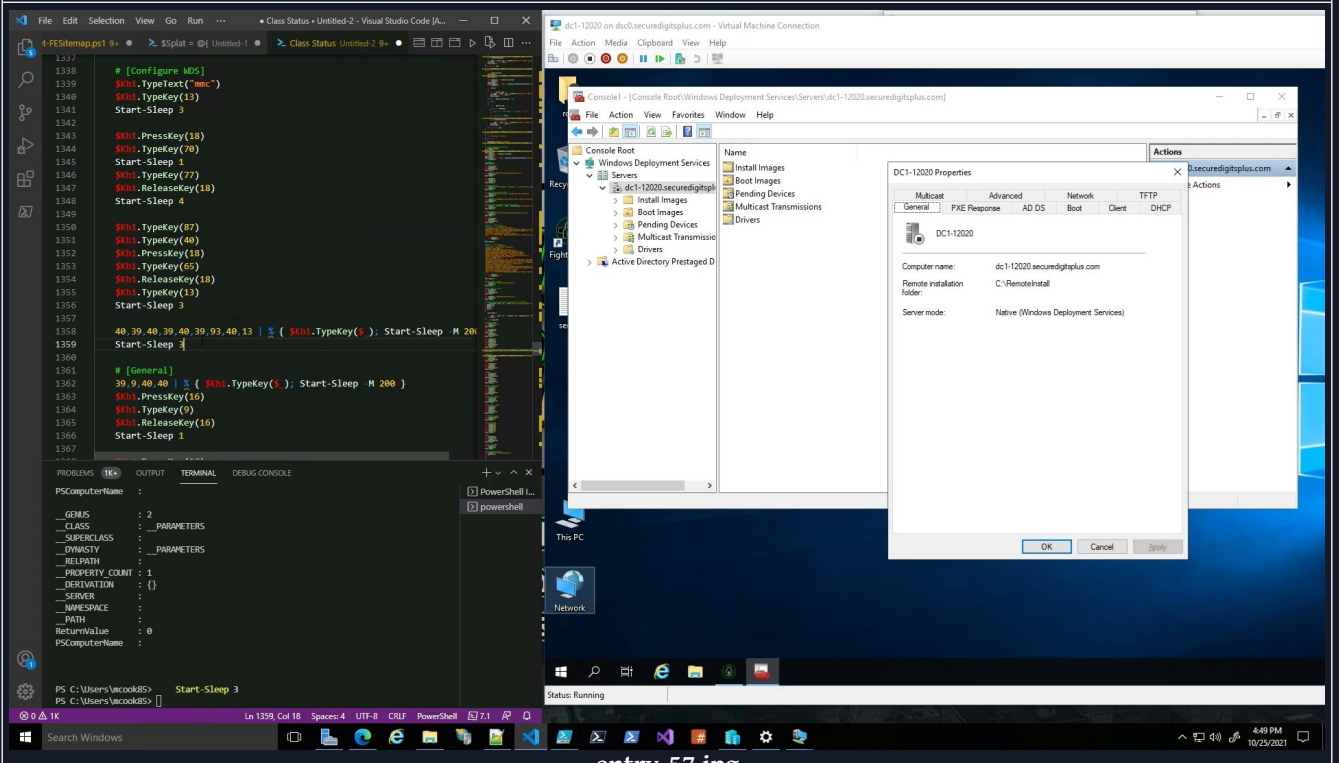
Alright, so at this point I was just testing stuff with the virtual machine controller, and wanted to use a combination of key presses to get to certain configuration interfaces on the system via (MMC/Microsoft Management Console).

Many of the things that [FightingEntropy(π)]://Infrastructure Deployment System does, will access things that MMC has access to, but also the registry, filesystem, certificate drive, IIS, etc. Depends on what it is.



entry-56.jpg

We'll wind up staging the WDS server right here, and then it'll be ready to prime additional child item systems.



entry-57.jpg

So, as I'm testing phases of the script on the left, it is feeding control from the bottom left console over the VM on the right. I know I've mentioned it, but I wanted to explain that all of those:

`40,39,40,39,40,93,40,13 | % { $Kb1.TypeKey($_) }`

statements are just pressing those keys on the <virtual machine>'s keyboard.

That is a ForEach-Object loop, that % is an alias for the command “ForEach-Object”.
Which is essentially the same thing as foreach, honestly.
There are differences between the 2, but they effectively do the same thing.

[3R]: FEInfrastructure (Preview/Demo)

Now, we can begin to (unpack/examine) the following video that shows off more of where Ambitious Automation left off:

12/05/21	FEInfrastructure Preview/Demo	1h 49m 57s	https://youtu.be/6yQr06_rA4I

This video shows a lot of other aspects of the [FightingEntropy(π)]://Infrastructure Deployment System.
With a bit of tinkering, this could wind up being a <billion dollar program>.

We start the scene off with some music by Megadeth, Peace Sells But Who’s Buying?
You can mute the music if you aren’t into that, the demonstration doesn’t have any commentary.